# PINAKA · SPEC OF-1 · ORDER FLOW READING OF CEILING AND FLOOR BREAKS

Draft 1 · Sun 04-Oct-2026 · NIFTY futures and ATM options via Fyers · Research → Spec → Python → Pine/Node

## INDEX


- 1 · PURPOSE AND SCOPE
  - 1.1 · What this spec is for
  - 1.2 · What it is not
  - 1.3 · Where it sits in the Pinaka corpus
- 2 · NOMENCLATURE — ORDER BOOK AND ORDER FLOW VOCABULARY
  - 2.1 · Order book terms
  - 2.2 · Trade and aggressor terms
  - 2.3 · Derived measures
  - 2.4 · Signature names (the eight)
  - 2.5 · Break states (from the Pinaka spec, unchanged)
- 3 · DATA — WHAT FYERS GIVES AND HOW TO SET IT UP
  - 3.1 · The two feeds
  - 3.2 · The 5-level setup
  - 3.3 · The 50-level (TBT) setup
  - 3.4 · Which instrument carries which signal
  - 3.5 · Symbols subscribed
  - 3.6 · The tick logger and its file format
  - 3.7 · History — why we log instead of download
- 4 · CODING CONTRACT — THE COMPUTATIONS
  - 4.1 · Tick rule
  - 4.2 · Price rows
  - 4.3 · Delta, CVD, row-Delta
  - 4.4 · Dot reference and dot sizes
  - 4.5 · Book measures
  - 4.6 · Tape speed and efficiency
  - 4.7 · Absorption clock
- 5 · SIGNATURE DETECTORS
  - 5.1 · ABS — absorption
  - 5.2 · EXH — exhaustion
  - 5.3 · DIV — delta divergence
  - 5.4 · IMP — impulse Delta
  - 5.5 · STK — stacked imbalance
  - 5.6 · PULL / STACK — pulling and stacking
  - 5.7 · ICE — iceberg
  - 5.8 · SPEED — tape speed
  - 5.9 · HOLLOW — spoof
  - 5.10 · WALL

- 6 · DISPLAYS
  - 6.1 · The main chart
  - 6.2 · The running-state box
  - 6.3 · The live ladder
  - 6.4 · The debug board
  - 6.5 · Legibility rules
- 7 · READING THE HEAT MAP — CASE CATALOGUE
- 8 · READING THE DOT CANDLES — CASE CATALOGUE
- 9 · COMBINED READING — HEAT AND DOTS TOGETHER
  - 9.1 · C1 ceiling → C BREAK ✓
  - 9.2 · C2 ceiling → failed START → flush
  - 9.3 · C3 floor → F BREAK ✓ (mirror)
  - 9.4 · What to do at each step
- 10 · TIME-BASED PATTERNS
  - 10.1 · Absorption clock
  - 10.2 · Tape speed
  - 10.3 · Refill rate
- 11 · THE TWO-WITNESS RULE — ATM CE AND ATM PE
  - 11.1 · Each witness on its own
  - 11.2 · The matrix
  - 11.3 · Order of turning
- 12 · LABEL LIST
- 13 · CONFIGURATION TABLE
- 14 · STUDY PROTOCOL — THE 20-SESSION TEST
- 15 · SCHEMATICS AND FIGURES
- 16 · PYTHON TWIN — orderflow_engine.py WITH PLAIN-ENGLISH NOTES
  - 16.1 · Block map
  - 16.2 · The source, verbatim
  - 16.3 · How to run it
- 17 · OPEN POINTS
- 18 · CHANGE REGISTER


---

# 1 · PURPOSE AND SCOPE

## 1.1 · What this spec is for

1. It defines how the NIFTY order book and order flow are read at a ceiling or a floor, so that a C BREAK START / C BREAK ✓ or an F BREAK START / F BREAK ✓ can be anticipated before the candle closes.
2. It fixes the vocabulary, the data setup on Fyers, every computation, every signature, every display, and the Python that produces them.
3. It is the contract for coding. What is not in this spec is not in the code.

## 1.2 · What it is not

1. It is not a strategy. Entries, stops and targets stay with the Pinaka intraday rulebook. This spec supplies the ORDER FLOW layer that the rulebook consults.
2. It is not a backtest result. §14 defines the test; the numbers come after 20 logged sessions.

## 1.3 · Where it sits in the Pinaka corpus

1. RESEARCH (this chat, Sun 04-Oct-2026) → SPEC OF-1 (this document) → PYTHON (`orderflow_engine.py`, §16) → PINE / NODE (derived later, display only).
2. Ceiling, floor, Vband, Eband, H str, H Prt, PARENTAGE SCORE, BREAKOUT SCORE and the four break states keep their Pinaka definitions. Nothing here redefines them.

---

# 2 · NOMENCLATURE — ORDER BOOK AND ORDER FLOW VOCABULARY

## 2.1 · Order book terms

| # | Term | Meaning |
|---|---|---|
| 1 | BID | A resting order to buy at a stated price. |
| 2 | ASK | A resting order to sell at a stated price. |
| 3 | BEST BID / BEST ASK | The highest bid and lowest ask at this instant. |
| 4 | SPREAD | Best ask − best bid. |
| 5 | LEVEL | One price with its total resting quantity. "5 levels" = the 5 best prices each side; "50 levels" = the 50 best. |
| 6 | DEPTH | The list of levels each side. Fyers gives 5 on the normal feed, 50 on the TBT feed (§3). |
| 7 | RESTING QTY | Quantity waiting at a level. It has not traded. |
| 8 | TOP-5 QTY | Sum of resting qty over the 5 best levels on one side. |
| 9 | TILT | Top-5 bid qty ÷ top-5 ask qty. Above 1 = book tilted to buyers. |
| 10 | WALL | One level whose qty is ≥ 3 × the median of its side. The visible ceiling or floor. |
| 11 | PULLING | Resting asks (or bids) cancelled before price reaches them. Measured as top-5 qty (wall excluded) falling to ≤ 50 % of its 10-minute average. |
| 12 | STACKING | Resting bids (or asks) being added. Measured as top-5 qty rising to ≥ 150 % of its 10-minute average. |
| 13 | REFILL | A level that was eaten by trades and comes back to ≥ 80 % of its earlier size. Counted per level. |
| 14 | ICEBERG | A level with ≥ 5 refills while price stays. A hidden order showing only part of itself. |
| 15 | HOLLOW | A large level that appeared and vanished ≥ 3 times with almost nothing traded there. Also called SPOOF. Not real support or resistance. |
| 16 | LADDER | The depth drawn vertically with the forming candle in the middle (§6.3, Fig 7 left). |
| 17 | HEAT | The depth over time painted behind the candles: green = bids, red = asks, darker = more (Fig 7 right, Fig 8). |

## 2.2 · Trade and aggressor terms

| # | Term | Meaning |
|---|---|---|
| 1 | LTP | Last traded price. |
| 2 | LTQ | Last traded quantity. |
| 3 | TICK | One trade message: time, LTP, LTQ, best bid, best ask. |
| 4 | AGGRESSOR | The side that crossed the spread to trade. Buyer aggressive = paid the ask. Seller aggressive = hit the bid. |
| 5 | TICK RULE | The classifier: LTP ≥ ask → buyer aggressive; LTP ≤ bid → seller aggressive; inside the spread → same side as the previous trade. An estimate; the exchange does not flag the aggressor. |
| 6 | BUY VOLUME | Sum of LTQ of buyer-aggressive trades in a candle. |
| 7 | SELL VOLUME | Sum of LTQ of seller-aggressive trades in a candle. |
| 8 | TAPE | The stream of trades. TAPE SPEED = trades per second. |

## 2.3 · Derived measures

| # | Term | Meaning |
|---|---|---|
| 1 | DELTA | Buy volume − sell volume for one candle. Positive = buyers pushed harder. |
| 2 | CVD | Cumulative Volume Delta: running sum of Delta from 09:15. A trend line of net pressure. |
| 3 | ROW | A 5-point price bucket (25175 covers 25175.00–25179.95). |
| 4 | ROW-DELTA | Delta computed per row inside one candle. The footprint. |
| 5 | DOT | The row with the highest row-Delta (green) and the row with the lowest (red) in a candle, drawn as a dot of one of three sizes (§4.4). |
| 6 | EFFICIENCY | Points moved ÷ Delta for a candle. Low at a ceiling = absorption; high = frictionless. |
| 7 | IMBALANCE | On one row, one side ≥ 3 × the other. |
| 8 | STACKED IMBALANCE | ≥ 3 consecutive rows imbalanced the same way. |
| 9 | REFERENCE | The rolling average used to size things: avg |

## 2.4 · Signature names (the eight)

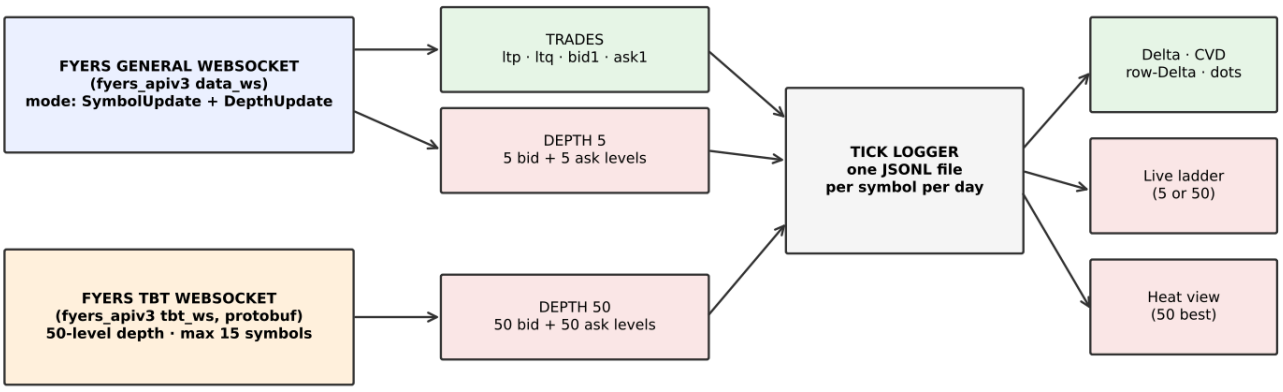
| # | Tag | Name | One line |
|---|---|---|---|
| 1 | ABS | Absorption | Big Delta, no progress. Sellers refilling. |
| 2 | EXH | Exhaustion | Delta shrinking at the same price. The push is tiring. |
| 3 | DIV | Delta divergence | Price higher high, CVD lower high (or mirror). Hollow. |
| 4 | IMP | Impulse Delta | Delta ≥ 3 × reference AND price moved. Someone big crossed. |
| 5 | STK | Stacked imbalance | ≥ 3 consecutive rows won by one side. |
| 6 | PULL / STACK | Pulling / stacking | The book stepping aside / being reinforced. |
| 7 | ICE | Iceberg | One level refilling again and again. |
| 8 | SPEED | Tape speed | Trades per second ≥ 2 × its 10-minute average. |

## 2.5 · Break states (from the Pinaka spec, unchanged)

1. C BREAK START — price gets beyond the ceiling at all (a wick poke by a red or doji counts). Printed once.
2. C BREAK ✓ — a full candle CLOSES above the ceiling, green. Printed once. State afterwards: ABOVE CEILING.
3. F BREAK START — price gets below the floor at all. Printed once.
4. F BREAK ✓ — a full candle CLOSES below the floor, red. Printed once. State afterwards: BELOW FLOOR.
5. A close back inside the band re-arms the tracker. There is no umbrella name over START and ✓.

# 3 · DATA — WHAT FYERS GIVES AND HOW TO SET IT UP

**S1 · DATA FEED SETUP — 5-level and 50-level depth**




**SYMBOLS SUBSCRIBED (3 of the 15 allowed): 1. NSE:NIFTY<MMM>FUT 2. rolling ATM CE 3. rolling ATM PE**

RULE: the 5-level feed is always on (trades come from it). The 50-level TBT feed is switched on when DEPTH_LEVELS_LOG = 50; when it is 5, the ladder and heat run on the 5 levels and the wall / pulling / stacking detectors still work, only coarser.

Mode names are as of Fyers API v3 (Sun 04-Oct-2026); confirm field names in the SDK before coding the connection.

## 3.1 · The two feeds

1. GENERAL WEBSOCKET (`fyers_apiv3` data socket). Message types: SymbolUpdate (LTP, LTQ, volume, best bid/ask, total buy qty, total sell qty) and DepthUpdate (5 bid levels + 5 ask levels with price and qty).
2. TBT WEBSOCKET (`fyers_apiv3` tick-by-tick socket, protobuf-encoded). Message type: depth with up to 50 levels each side. Up to 15 symbols per subscription. Industry-first on Fyers as of 2025; confirm it is enabled on the account.
3. Trades always come from feed 1. Depth comes from feed 1 (5 levels) or feed 2 (50 levels) according to `DEPTH_LEVELS_LOG`.

## 3.2 · The 5-level setup

1. Create the data socket with the access token; subscribe the three symbols of §3.5 with data type `SymbolUpdate` and `DepthUpdate`.
2. On every SymbolUpdate call `logger.on_trade(symbol, ts, ltp, ltq, bid1, ask1)`.
3. On every DepthUpdate call `logger.on_depth(symbol, ts, bids5, asks5)`.
4. Set `DEPTH_LEVELS_LOG = 5`. The ladder, heat, wall, pulling, stacking and iceberg detectors all run on 5 levels — coarser but valid.

## 3.3 · The 50-level (TBT) setup

1. Create the TBT socket with the same token; subscribe the same symbols for depth.
2. Decode the protobuf message to lists of `[price, qty]` best-first; call `logger.on_depth(symbol, ts, bids50, asks50)`.
3. Set `DEPTH_LEVELS_LOG = 50`. Keep `DEPTH_LEVELS_HEAT = 10` so the heat view draws only the 10 best levels each side; the remaining 40 feed the wall / hollow detectors and the ladder.

4. Rule: the heat must stay light. More than 10 rows each side turns it into wallpaper and the dots drown (Fig 8 is drawn at 75 % strength for this reason).

## 3.4 · Which instrument carries which signal

| # | Signal | Instrument | Why |
|---|---|---|---|
| 1 | Heat, ladder, wall, pulling, stacking, iceberg, hollow | NIFTY FUT | Deepest book that trades index direction; option books are thin and quote-driven. |
| 2 | Delta, CVD, dots, ABS, EXH, IMP, STK, DIV, SPEED | NIFTY FUT | Primary. |
| 3 | Witness UP / DOWN | ATM CE and ATM PE, each on its own | §11. Options volume dwarfs futures; aggression may print there first. |
| 4 | Nothing | NIFTY 50 index | It is a calculation, not a security: no book, no trades. Its "volume" is the sum of 50 stock volumes. May be shown as the price line only. |

## 3.5 · Symbols subscribed

1. `NSE:NIFTY<MMM>FUT` — current month; roll on expiry day after 15:30.
2. Rolling ATM CE — strike = nearest 50 to FUT LTP; re-pick when LTP crosses ±25 of the current strike; the Delta series is stitched across the change (§4.3 point 5).
3. Rolling ATM PE — same rule.
4. Three symbols; 12 of the 15 TBT slots remain free.

## 3.6 · The tick logger and its file format

1. One JSONL file per symbol per day: `NSE_NIFTY26OCTFUT_2026-10-01.jsonl`.
2. Trade line: `{"t":"T","ts":<epoch s>,"p":<ltp>,"q":<ltq>,"b":<bid1>,"a":<ask1>}`.
3. Depth line: `{"t":"D","ts":<epoch s>,"bids":[[p,q],…],"asks":[[p,q],…]}` best-first.
4. The file is replayable: `python orderflow_engine.py <file> <ceiling> <out.png>` rebuilds every candle, tag and chart exactly as they were seen live. The study (§14) runs on these files.
5. Size: NIFTY FUT at 50 levels ≈ 150–300 MB per day. Keep 30 days hot, compress older.

## 3.7 · History — why we log instead of download

1. Fyers history API returns candles only (OHLCV). No historical ticks, no historical depth.
2. Historical tick data with best bid/ask can be bought (NSE Data & Analytics, GDFL, TrueData); full 50-level book history is expensive.
3. Decision: log from the first day; buy history only if the first 20 sessions justify it (§14).

---

# 4 · CODING CONTRACT — THE COMPUTATIONS

## 4.1 · Tick rule

1. For each trade: if LTP ≥ best ask → side = +1 (buyer aggressive). If LTP ≤ best bid → side = −1. Otherwise side = the previous trade's side (`MID_TRADE_RULE = "last"`).
2. Accuracy is roughly 85–90 %; the misclassified remainder is noise that averages out over a candle. CVD is therefore a trend read, not an exact count.

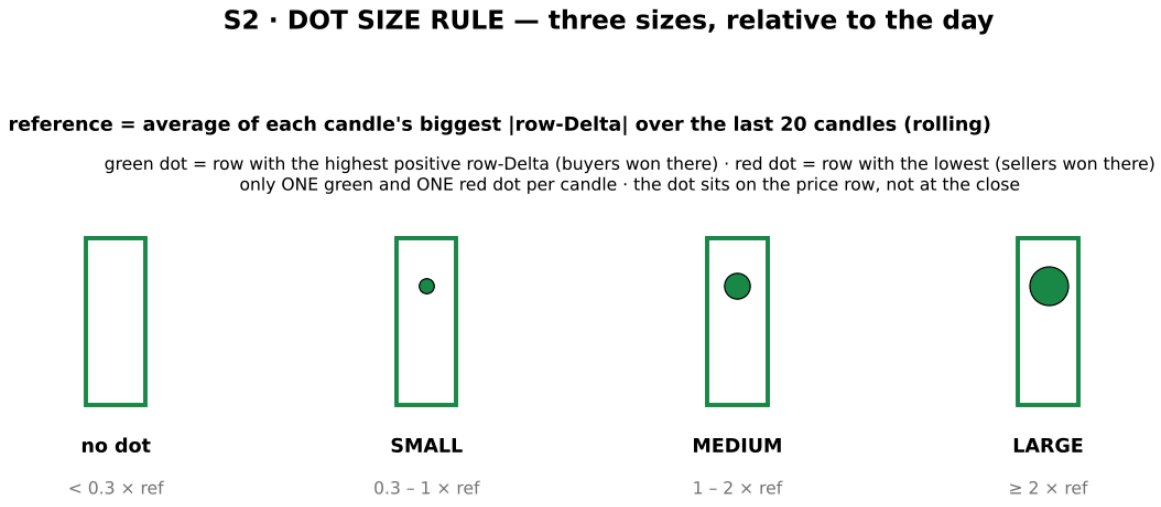
## 4.2 · Price rows

1. `row(price) = floor(price / ROW_SIZE) × ROW_SIZE` with `ROW_SIZE = 5`.
2. Every trade is credited to one row; every depth level is drawn on one row.

## 4.3 · Delta, CVD, row-Delta

1. Candle Delta = Σ LTQ (side +1) − Σ LTQ (side −1) over the candle.
2. CVD = running Σ Delta from the first candle of the session.
3. Row-Delta = the same per row inside the candle.
4. Reference for signatures = mean |Delta| over the previous 20 completed candles (`AVG_DELTA_WINDOW`), never less than 1.
5. Options stitch: when the ATM strike changes, the new strike's CVD starts at the old strike's last CVD value, so the witness line does not jump.

## 4.4 · Dot reference and dot sizes




1. Dot reference = mean, over the previous 20 completed candles, of each candle's largest |row-Delta| (`DOT_REF_WINDOW`).
2. Green dot = the row with the highest positive row-Delta. Red dot = the row with the lowest negative row-Delta. At most one of each per candle.
3. Size: < 0.3 × ref → no dot; 0.3–1 × → SMALL; 1–2 × → MEDIUM; ≥ 2 × → LARGE (`DOT_FLOOR`, `DOT_MED`, `DOT_LARGE`).
4. The dot sits on its price row, not at the close. Sizes are relative to the day, so an expiry day and a quiet day read the same way.

## 4.5 · Book measures

1. Top-5 qty per side; tilt = bid top-5 ÷ ask top-5.
2. Wall = level ≥ `WALL_X` (3) × the median qty of its side.
3. Pulling = top-5 ask qty with the wall level excluded ≤ `PULL_DROP` (0.5) × its `BOOK_AVG_MIN` (10) minute average. Mirror for bids at a floor.
4. Stacking = top-5 bid qty ≥ `STACK_RISE` (1.5) × its 10-minute average. Mirror for asks at a floor.
5. Refill count per level: +1 each time a level that was big (≥ `SPOOF_MIN_QTY_X` (2) × median) is traded down by ≥ 80 % and returns to ≥ 80 % of its earlier size.
6. Hollow count per level: +1 each time a big level disappears with < 20 % of its size traded.

## 4.6 · Tape speed and efficiency

1. Tape speed = trades ÷ `CANDLE_SEC` for the candle; reference = mean over the previous 10 candles.
2. Efficiency = |close − open| ÷ max(|Delta|, 50). Used in the heat-line display (Fig 5 panel 2) and in the study, not as a detector.

## 4.7 · Absorption clock

1. Starts at the first candle that prints ABS under the ceiling (or over the floor).
2. Zones: EARLY < `ABS_CLOCK_GREEN_MIN`; GREEN until `ABS_CLOCK_RED_MIN`; RED after. Both thresholds are OPEN and set from the study (§14).
3. Resets on C BREAK ✓ or F BREAK ✓.

---

# 5 · SIGNATURE DETECTORS

Each detector is one Python function in §16 returning (fired, value). Nothing prints before WARMUP_CANDLES (5) completed candles.

## 5.1 · ABS — absorption

1. Fires when |Delta| ≥ ABS_DELTA_X (1.5) × reference AND |close − open| ≤ ABS_PROGRESS_ROWS (1) row.
2. Meaning: aggressive volume consumed by resting orders without moving price. At a ceiling it is sellers refilling; at a floor it is buyers.

## 5.2 · EXH — exhaustion

1. Fires when the last EXH_CANDLES + 1 (3) candles close within one row of each other, their Deltas have the same sign, each |Delta| is smaller than the one before, and the last is ≤ EXH_SHRINK (0.5) × the first.
2. Meaning: the pushing side is running out at that price.

## 5.3 · DIV — delta divergence

1. Fires when, over the last DIV_LOOKBACK (5) candles vs the 5 before, price makes a high ≥ 1 row higher while CVD makes a lower high (bear), or price a low ≥ 1 row lower while CVD a higher low (bull).
2. Meaning: the move is not backed by aggression — hollow.

## 5.4 · IMP — impulse Delta

1. Fires when |Delta| ≥ IMP_DELTA_X (3) × reference AND |close − open| ≥ IMP_PROGRESS_ROWS (2) rows.
2. Meaning: a large participant crossed the spread and price followed. The ✓ signature when it lands on the break candle.

## 5.5 · STK — stacked imbalance

1. For each row in the candle: buy imbalance if buy ≥ STK_RATIO (3) × sell; sell imbalance if the reverse. Fires when ≥ STK_ROWS (3) consecutive rows are imbalanced the same way.
2. Meaning: one side lifted every level in sequence — the order-flow form of a wide-range body.

## 5.6 · PULL / STACK — pulling and stacking

1. PULL fires per §4.5 point 3; STACK per §4.5 point 4. Evaluated every depth update, reported per candle.
2. Meaning: the book stepping aside ahead of price (PULL) or reinforcing under it (STACK). PULL above a ceiling + STACK below price = the book giving way.

## 5.7 · ICE — iceberg

1. Fires for a level with refill count ≥ ICE_REFILLS (5) while price has not closed beyond that level.
2. Meaning: a hidden order defending that price. Breaks into an iceberg usually fail until its refills slow (§10.3).

## 5.8 · SPEED — tape speed

1. Fires when tape speed ≥ SPEED_X (2) × its 10-candle reference.
2. Meaning: momentum of participation. Breaks that hold tend to have SPEED on the ✓ candle; failed STARTs are slow pokes.

## 5.9 · HOLLOW — spoof

1. Fires for a level with hollow count ≥ SPOOF_VANISH (3).
2. Meaning: displayed size that never intended to trade. A hollow bid under price is not support; a hollow ask on the ceiling is not resistance.

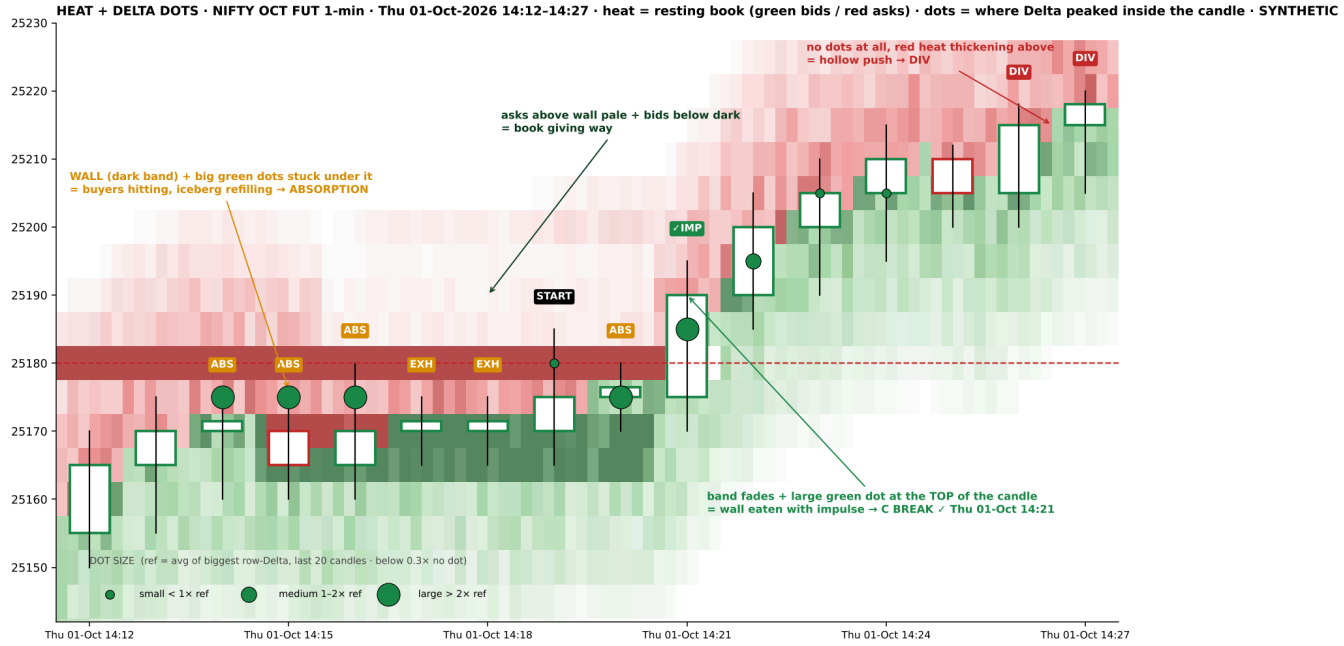
## 5.10 · WALL

1. Reported, not fired: the largest level per side when it is ≥ WALL_X × median. Drawn as the dark band in the heat and the long bar in the ladder.

---

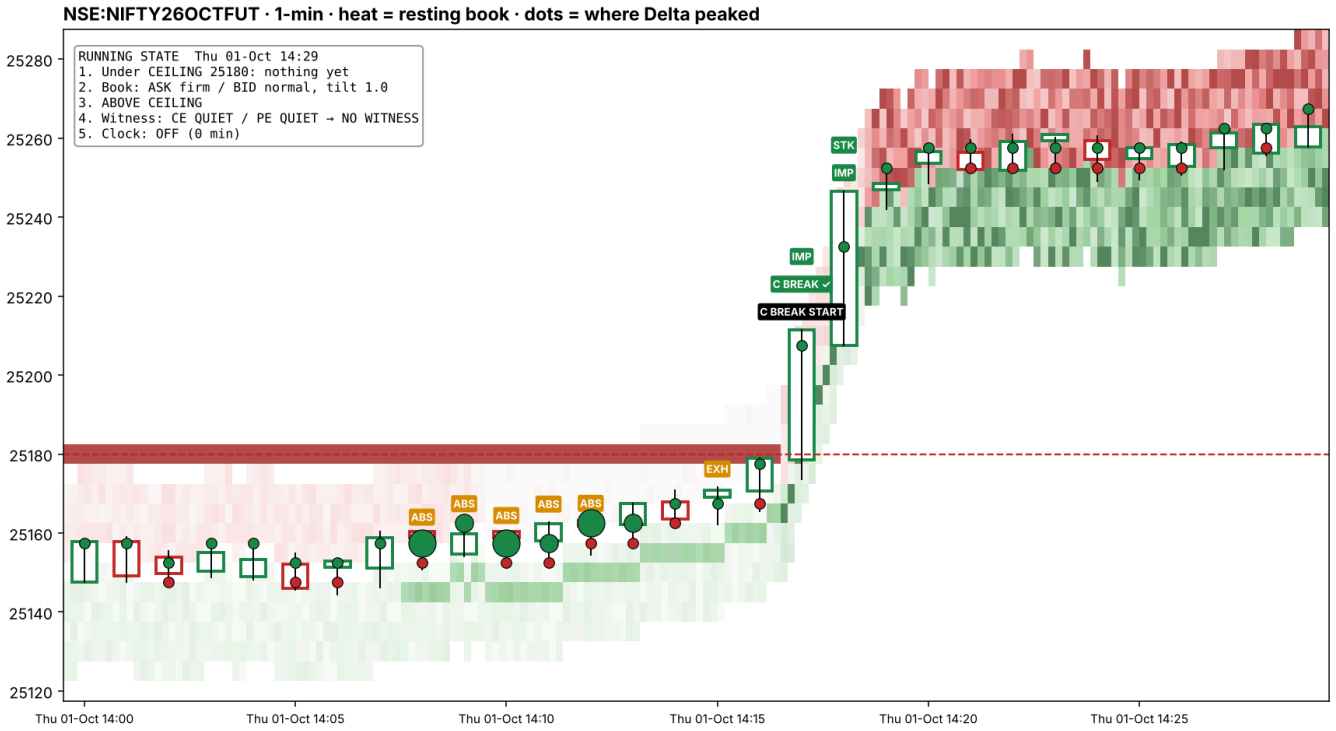
# 6 · DISPLAYS

## 6.1 · The main chart




1. Background: HEAT — resting book over time, 10 best levels each side, green bids / red asks, darker = more, drawn at 75 % strength.
2. Candles: hollow bodies with the price colour on the outline, so the heat shows through.
3. Dots: one green and one red per candle at most, three sizes (§4.4).
4. Tags: one word per fired signature above the candle, at most three per candle; break events (START, ✓) print first.
5. Ceiling and floor as dashed lines, from the Pinaka engine.
6. Every x-axis label carries day, date and time: "Thu 01-Oct 14:21".

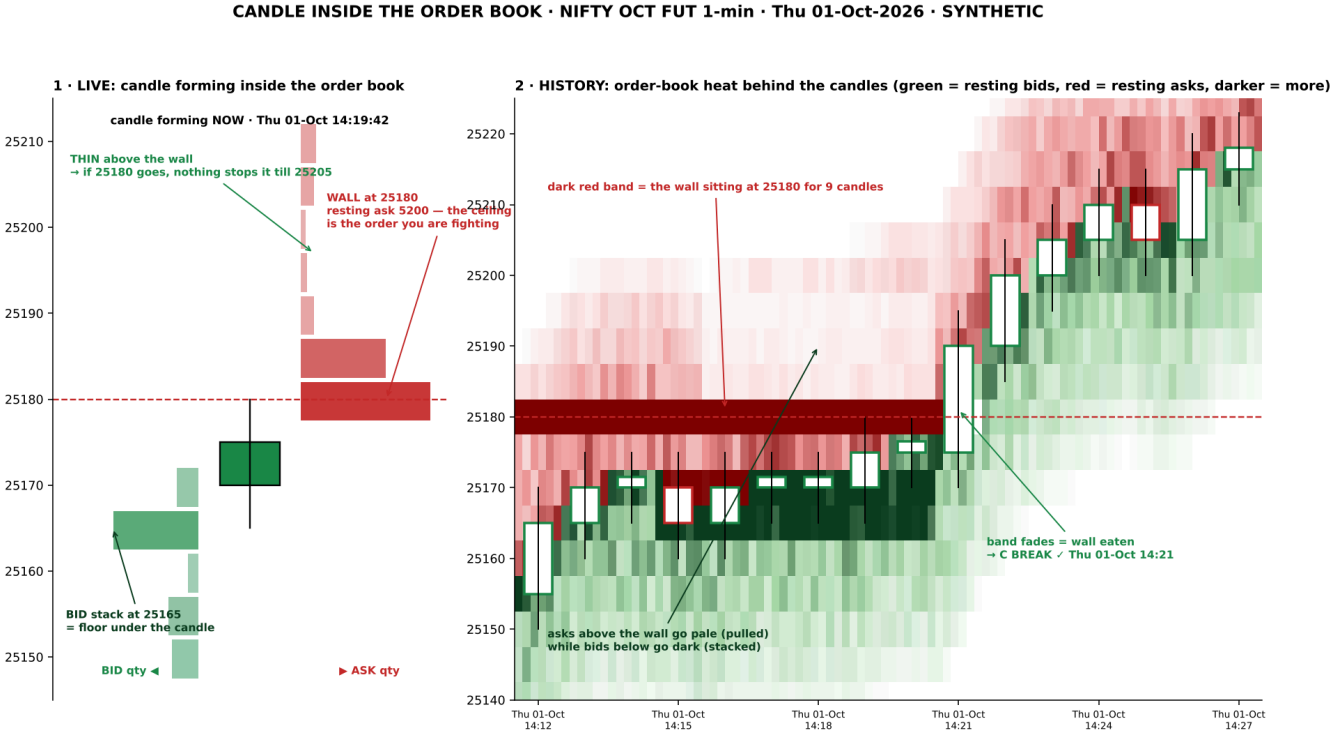
Fig 9 is the same chart produced by the Python twin itself from a logged file (synthetic day, Thu 01-Oct-2026 14:00–14:29): ABS tags under the wall, EXH, then IMP + STK + C BREAK START + C BREAK ✓ on the break candle.




## 6.2 · The running-state box

1. Five lines, nothing more:
    1. Under CEILING / over FLOOR: the signature chain seen so far (e.g. "ABS → EXH → START").
    2. Book: ASK thin / firm, BID stacked / normal, tilt.
    3. Break state: inside band / START seen, no ✓ / ABOVE CEILING / BELOW FLOOR.
    4. Witness: CE and PE verdicts and the matrix result (§11).
    5. Clock zone and minutes (§10.1).
2. Monospace, top-left, always visible.

## 6.3 · The live ladder



1. The forming candle in the middle; bid qty as green bars to the left, ask qty as red bars to the right, bar length = qty.
2. The wall is the longest bar; what is above / below it is read directly ("thin above → frictionless once the wall goes").
3. Runs on 5 or 50 levels; on 50 the ladder is scrolled to ±15 levels around price.

## 6.4 · The debug board




1. Delta bars, CVD line, top-5 bid/ask qty, tilt ratio, and the numeric footprint (Fig 3) live here. Optional, for analysis after the session, never on the trading screen.

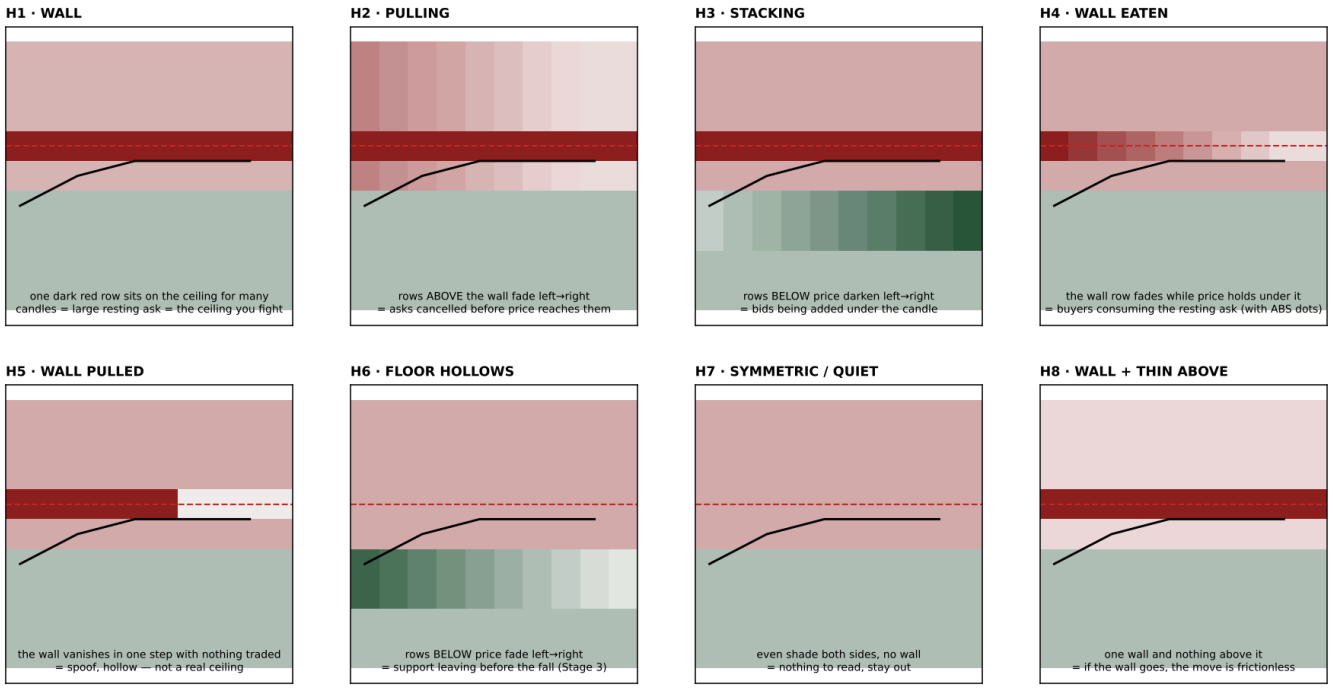
## 6.5 · Legibility rules

1. No numbers on the main chart. Shade, size and one word only.
2. At most three tags per candle; at most five lines in the running-state box.
3. Heat at 75 % strength, 10 levels each side.
4. Day, date and time on every chart reference.

---

# 7 · READING THE HEAT MAP — CASE CATALOGUE

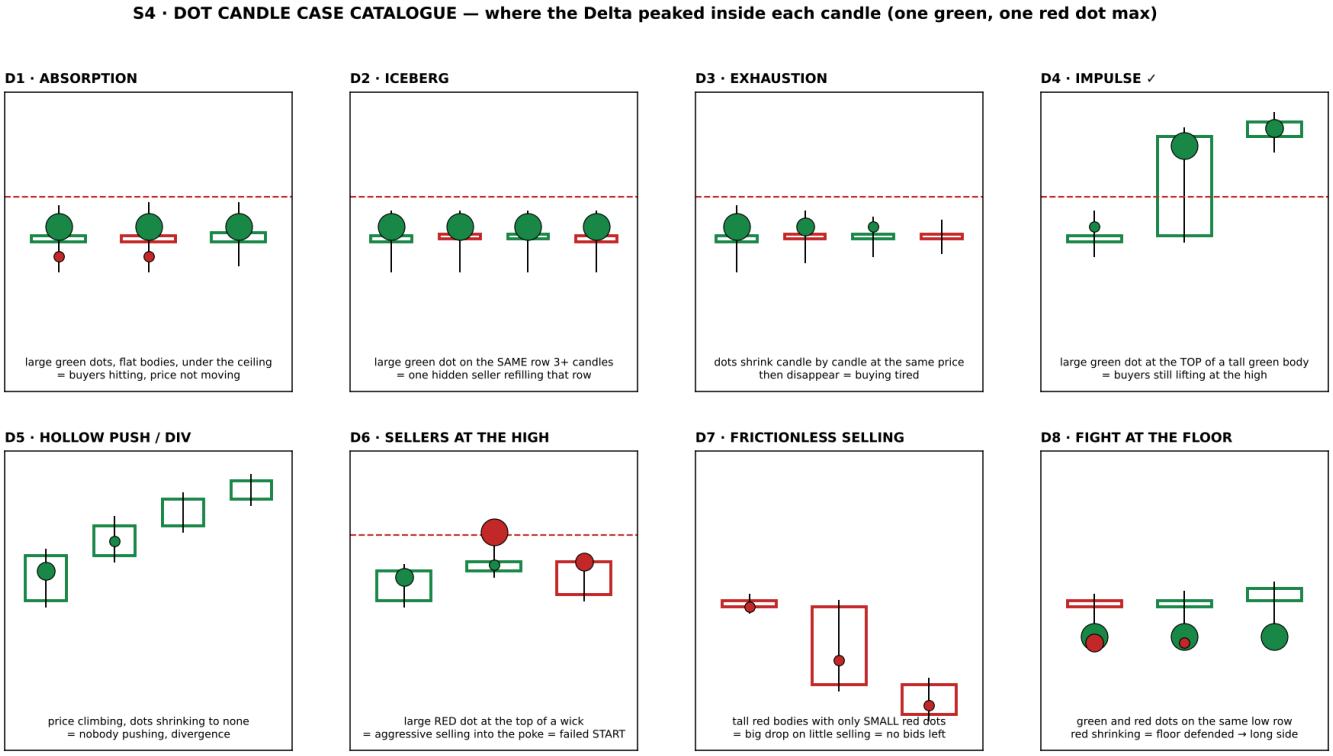
**S3 · HEAT CASE CATALOGUE — what each pattern in the resting book looks like (green = bids, red = asks, darker = more; black line = price)**




| # | Case | What you see | What it means | What to do |
|---|---|---|---|---|
| H1 | WALL | One dark red row on the ceiling across many candles. | Large resting ask; this is the ceiling you fight. | Expect ABS. No CE until the wall fades or is pulled. |
| H2 | PULLING | Rows above the wall fade left → right. | Asks cancelled before price reaches them. | Stage 2 of the book giving way. Watch for H4. |
| H3 | STACKING | Rows below price darken left → right. | Bids being added under the candle. | Floor firming; combined with H2 it is pre-break. |
| H4 | WALL EATEN | The wall row fades while price holds under it, with ABS dots. | Buyers consuming the resting ask. | Clock green: ✓ is near. CE allowed on C BREAK ✓. |
| H5 | WALL PULLED | The wall vanishes in one step with nothing traded. | Spoof; the ceiling was hollow. | Do not treat it as a win for buyers; wait for IMP. |
| H6 | FLOOR HOLLOWS | Rows below price fade left → right. | Support leaving before the fall (Stage 3). | No CE. Prepare for F BREAK START. |
| H7 | SYMMETRIC / QUIET | Even shade both sides, no wall. | Nothing to read. | Stay out. |
| H8 | WALL + THIN ABOVE | One wall and nothing above it. | If the wall goes, the move is frictionless. | Highest-reward ✓ setup; size for the run. |

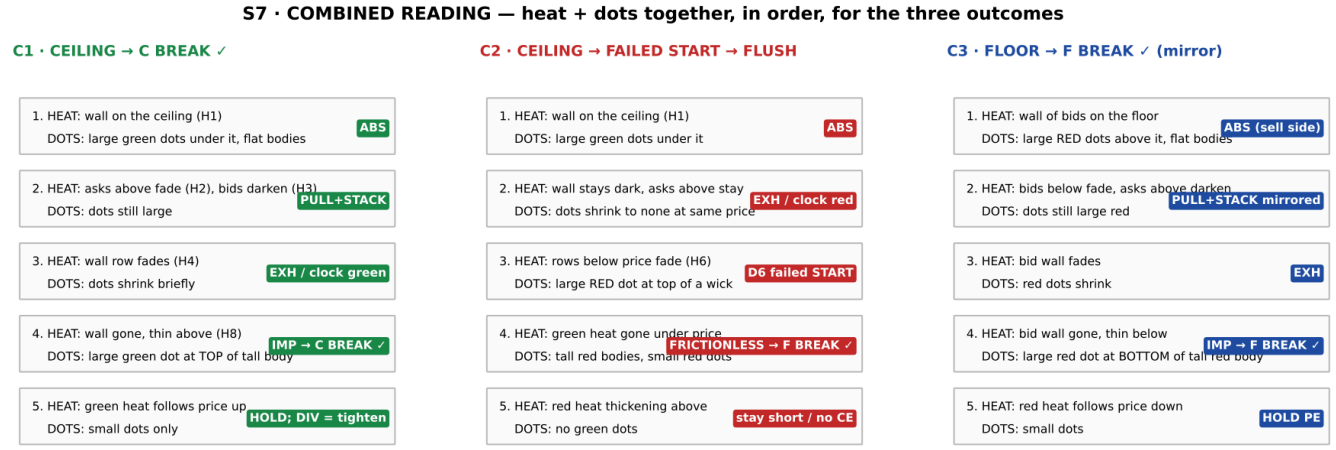
Mirror every case for a floor: swap bids for asks, green for red, "above" for "below".

# 8 · READING THE DOT CANDLES — CASE CATALOGUE




| # | Case | What you see | What it means | What to do |
|---|---|---|---|---|
| D1 | ABSORPTION | Large green dots, flat bodies, under the ceiling. | Buyers hitting, price not moving. | Read the heat: H4 → ✓ likely; H1 holding → wait. |
| D2 | ICEBERG | Large green dot on the same row for 3+ candles. | One hidden seller refilling that row. | No CE while refills continue (§10.3). |
| D3 | EXHAUSTION | Dots shrink candle by candle at the same price, then vanish. | Buying tired. | If H6 appears next → failed START; if H4 → wall nearly gone. |
| D4 | IMPULSE ✓ | Large green dot at the top of a tall green body. | Buyers still lifting at the high. | C BREAK ✓ with IMP = entry per rulebook. |
| D5 | HOLLOW PUSH | Price climbing, dots shrinking to none. | Nobody pushing; DIV. | Tighten stop; no fresh CE. |
| D6 | SELLERS AT THE HIGH | Large red dot at the top of a wick. | Aggressive selling into the poke. | Failed START; expect flush. |
| D7 | FRICTIONLESS SELLING | Tall red bodies with only small red dots. | Big drop on little selling; no bids left. | Short side only; this is the Spec 3 frictionless wave. |
| D8 | FIGHT AT THE FLOOR | Green and red dots on the same low row, red shrinking. | Floor defended. | Long side per the CE-at-the-floor rule. |

# 9 · COMBINED READING — HEAT AND DOTS TOGETHER




## 9.1 · C1 ceiling → C BREAK ✓

1. H1 + D1 → ABS.
2. H2 + H3, dots still large → PULL + STACK.
3. H4, dots shrink briefly → EXH, clock green.
4. H8 (wall gone, thin above) + D4 → IMP → C BREAK ✓.
5. Green heat follows price, small dots → hold; DIV = tighten.

## 9.2 · C2 ceiling → failed START → flush

1. H1 + D1 → ABS.
2. Wall stays dark, D3 → EXH, clock red.
3. H6 + D6 → failed START.
4. Green heat gone under price + D7 → frictionless → F BREAK ✓.
5. Red heat thickening above, no green dots → stay short, no CE.

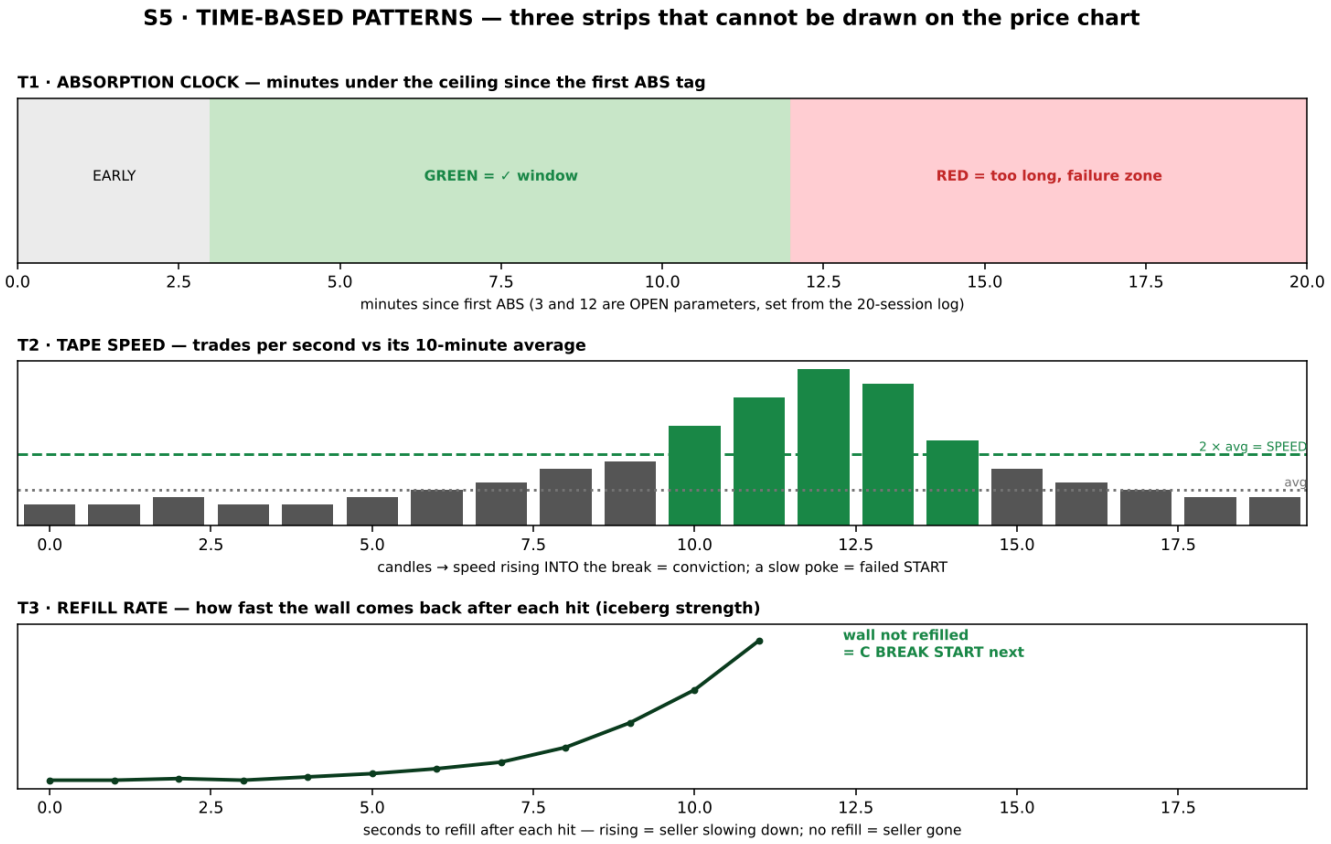
## 9.3 · C3 floor → F BREAK ✓ (mirror)

1. Bid wall on the floor + large red dots → ABS (sell side).
2. Bids below fade, asks above darken → PULL + STACK mirrored.
3. Bid wall fades, red dots shrink → EXH.
4. Bid wall gone, thin below + large red dot at the bottom of a tall red body → IMP → F BREAK ✓.
5. Red heat follows price down, small dots → hold PE.

## 9.4 · What to do at each step

1. ABS alone = wait. ABS + PULL + STACK = prepare. ABS + H4 + clock green = ✓ expected. ✓ + IMP + (SPEED) = act per rulebook. DIV after ✓ = tighten.
2. ABS + clock red + D3 = failure expected. H6 or D6 = no CE. D7 = short side.
3. The two-witness result (§11) must agree with the direction, or the step is "prepare" only.

# 10 · TIME-BASED PATTERNS




## 10.1 · Absorption clock

1. Counts minutes since the first ABS under the ceiling. Three zones (§4.7). Shown in the running-state box, not on the chart.
2. The 20-session log gives the typical minutes-under-ceiling before ✓ and before failure; the zone boundaries are set from those two distributions.

## 10.2 · Tape speed

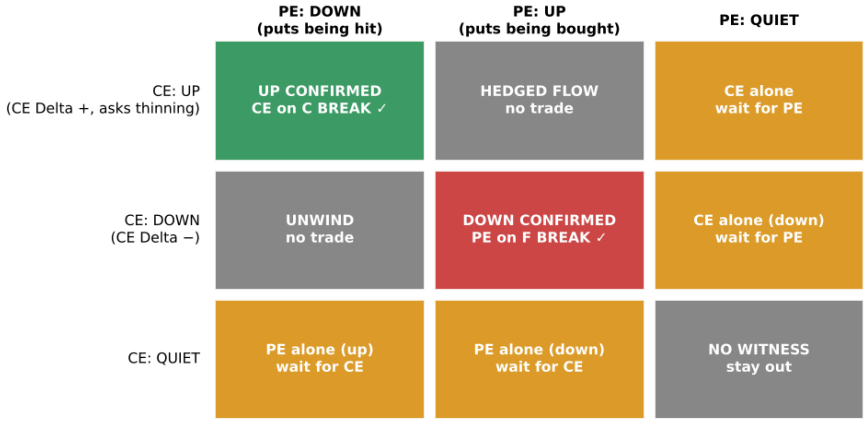
1. Trades per second per candle vs its 10-candle average. SPEED fires at 2 ×.
2. Speed rising into the break = conviction; a slow poke = failed START.

## 10.3 · Refill rate

1. Seconds between a wall being hit and refilled, plotted per hit.
2. Rising refill time = the seller slowing; no refill = seller gone → C BREAK START next. This is the iceberg's own exhaustion.

# 11 · THE TWO-WITNESS RULE — ATM CE AND ATM PE

**S6 · TWO-WITNESS MATRIX — ATM CE book and ATM PE book read separately, then compared**




UP needs CALL BUYERS aggressive AND PUTS being dumped. Both being bought = volatility bet, not direction. The FIRST witness to turn is the warning, the second is the confirmation.

## 11.1 · Each witness on its own

1. CE witness UP: CE Delta ≥ 1 × its reference and (CE asks pulling or price progress ≥ 1 row). CE witness DOWN: CE Delta ≤ −1 × reference. Else QUIET.
2. PE witness: same rule on the PE book. PE DOWN (puts being hit) is the bullish witness; put selling is the bullish act.

## 11.2 · The matrix

| CE \ PE | PE DOWN | PE UP | PE QUIET |
|---|---|---|---|
| CE UP | UP CONFIRMED — CE on C BREAK ✓ | HEDGED FLOW — no trade | CE alone — wait |
| CE DOWN | UNWIND — no trade | DOWN CONFIRMED — PE on F BREAK ✓ | CE alone (down) — wait |
| CE QUIET | PE alone (up) — wait | PE alone (down) — wait | NO WITNESS |

## 11.3 · Order of turning

1. The first witness to turn is the warning; the second turning is the confirmation.
2. The log records which witness led on every ✓; that ordering is itself a signature for the study.

# 12 · LABEL LIST

| # | Label | Where it prints | Fires when |
|---|---|---|---|
| 1 | ABS | tag above candle | §5.1 |
| 2 | EXH | tag above candle | §5.2 |
| 3 | DIV | tag above candle | §5.3 |
| 4 | IMP | tag above candle | §5.4 |
| 5 | STK | tag above candle | §5.5 |
| 6 | SPEED | tag above candle | §5.8 |
| 7 | PULL | running-state line 2 ("ASK thin") | §5.6 |
| 8 | STACK | running-state line 2 ("BID stacked") | §5.6 |
| 9 | ICE | ladder row marker + heat band label | §5.7 |
| 10 | HOLLOW | hatched ladder row | §5.9 |
| 11 | WALL | dark heat band / longest ladder bar | §5.10 |
| 12 | C BREAK START | tag, once | §2.5 |
| 13 | C BREAK ✓ | tag, once | §2.5 |
| 14 | F BREAK START | tag, once | §2.5 |
| 15 | F BREAK ✓ | tag, once | §2.5 |
| 16 | green dot S/M/L | on its row inside the candle | §4.4 |
| 17 | red dot S/M/L | on its row inside the candle | §4.4 |
| 18 | UP CONFIRMED / DOWN CONFIRMED / HEDGED FLOW / UNWIND / NO WITNESS | running-state line 4 | §11.2 |
| 19 | EARLY / GREEN / RED / OFF | running-state line 5 | §4.7 |

# 13 · CONFIGURATION TABLE

| # | Parameter | Value | Owner | Used by |
|---|---|---|---|---|
| 1 | SYMBOL_FUT | NSE:NIFTYFUT | Tushar | §3.5 |
| 2 | ATM_STEP | 50 | NSE | §3.5 |
| 3 | DEPTH_LEVELS_LOG | 50 (or 5) | Tushar | §3.2–3.3 |
| 4 | DEPTH_LEVELS_HEAT | 10 | Claude | §6.1 |
| 5 | CANDLE_SEC | 60 | Tushar | §4.3 |
| 6 | ROW_SIZE | 5 | Claude | §4.2 |
| 7 | MID_TRADE_RULE | last | Claude | §4.1 |
| 8 | DOT_REF_WINDOW | 20 | Claude | §4.4 |
| 9 | DOT_FLOOR / DOT_MED / DOT_LARGE | 0.3 / 1.0 / 2.0 | Claude | §4.4 |
| 10 | ABS_DELTA_X / ABS_PROGRESS_ROWS | 1.5 / 1 | Claude | §5.1 |
| 11 | EXH_CANDLES / EXH_SHRINK | 2 / 0.5 | Claude | §5.2 |
| 12 | IMP_DELTA_X / IMP_PROGRESS_ROWS | 3.0 / 2 | Claude | §5.4 |
| 13 | STK_RATIO / STK_ROWS | 3.0 / 3 | Claude | §5.5 |
| 14 | DIV_LOOKBACK | 5 | Claude | §5.3 |
| 15 | ICE_REFILLS / ICE_REFILL_RATIO | 5 / 0.8 | Claude | §5.7 |
| 16 | SPOOF_VANISH / SPOOF_MIN_QTY_X | 3 / 2.0 | Claude | §5.9 |
| 17 | PULL_DROP / STACK_RISE / BOOK_AVG_MIN | 0.5 / 1.5 / 10 | Claude | §5.6 |
| 18 | WALL_X | 3.0 | Claude | §5.10 |
| 19 | SPEED_X | 2.0 | Claude | §5.8 |
| 20 | AVG_DELTA_WINDOW | 20 | Claude | §4.3 |
| 21 | WARMUP_CANDLES | 5 | Claude | §5 |
| 22 | ABS_CLOCK_GREEN_MIN / ABS_CLOCK_RED_MIN | 3 / 12 | OPEN — set from §14 | §4.7 |
| 23 | BREAK_NEEDS_BODY_CLOSE | True | Tushar | §2.5 |

Every "Claude" value is a starting point to be retuned from the 20-session log; the study report proposes the new value and the owner changes to Tushar on acceptance.

## 14 · STUDY PROTOCOL — THE 20-SESSION TEST

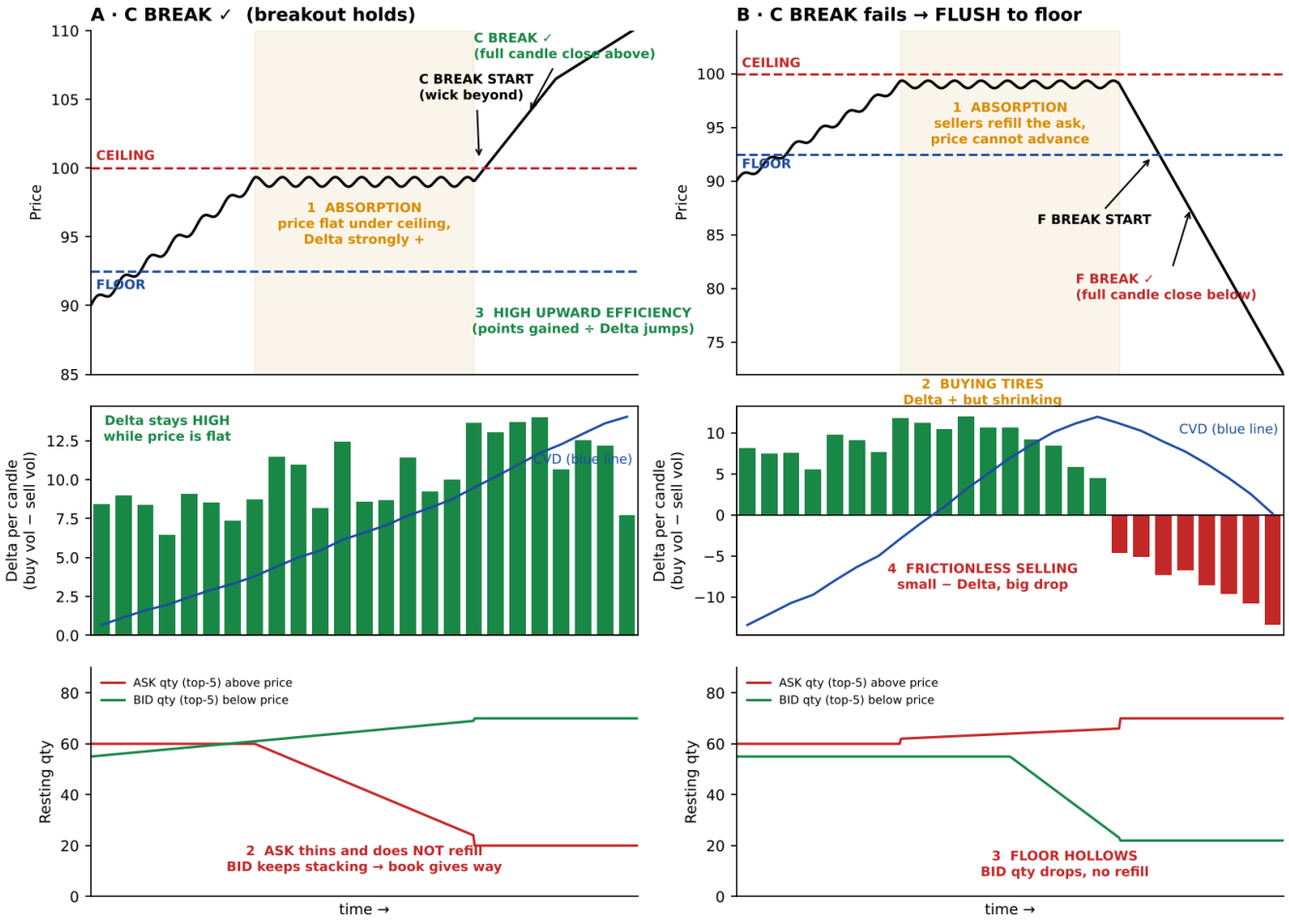
1. Run the logger every session from the first day; three symbols; both feeds.
2. Each evening, replay the FUT file with that day's ceiling and floor from the Pinaka engine; the replay writes one row per candle: time, Delta, CVD, dots, every detector's (fired, value), book measures, witnesses, clock zone, break events.
3. Outcome labels per ceiling test: ✓ HELD (closed above and did not close back inside within 15 candles), ✓ FAILED (closed above then back inside), START ONLY (poked, never closed), NO START.
4. For each signature and for the chain ABS → PULL+STACK → EXH → IMP, count presence in the 15 candles before each outcome. Report the ratio of ✓ HELD to all other outcomes with the signature present vs absent.
5. A signature earns a place in the trading rule when ✓ HELD is at least 1.5 × more frequent with it present than absent over ≥ 15 tests. Otherwise it stays in the debug board.
6. Compare FUT-only against FUT + two-witness; record which witness led on every ✓.
7. Set ABS_CLOCK_GREEN_MIN and ABS_CLOCK_RED_MIN from the minutes-under-ceiling distributions of ✓ HELD vs failures.
8. Deliverable: a one-page table plus the retuned configuration, as draft 2 of this spec.

---

# 15 · SCHEMATICS AND FIGURES

1. Fig 1 — Concept: breakout vs failure as drawn paths (price, Delta/CVD, resting book). Research figure.
2. Fig 2 — Debug board layout (synthetic, Thu 01-Oct-2026 13:31–14:30).
3. Fig 3 — Numeric footprint (synthetic, Thu 01-Oct-2026 14:12–14:27). Debug only.
4. Fig 4 — Colour-only trading view, superseded by Fig 8.
5. Fig 5 — Six innovation mock-ups (pressure glyph, efficiency heat line, tug-of-war / clock / gauge, iceberg ladder + spoof, signature sentence + DNA strip, phase-aware tag / options lamp / teach mode). Candidate layers, deferred (§17).
6. Fig 6 — Three ways of drawing Delta inside the candle; variant C (dots) adopted.
7. Fig 7 — Candle inside the order book: live ladder and heat history.
8. Fig 8 — The main chart: heat + dots + tags.
9. Fig 9 — The same chart produced by the Python twin from a logged file.
10. S1 — Data feed setup, 5 and 50 levels.
11. S2 — Dot size rule.
12. S3 — Heat case catalogue, H1–H8.
13. S4 — Dot candle case catalogue, D1–D8.
14. S5 — Time strips: clock, tape speed, refill rate.
15. S6 — Two-witness matrix.
16. S7 — Combined sequences C1–C3.

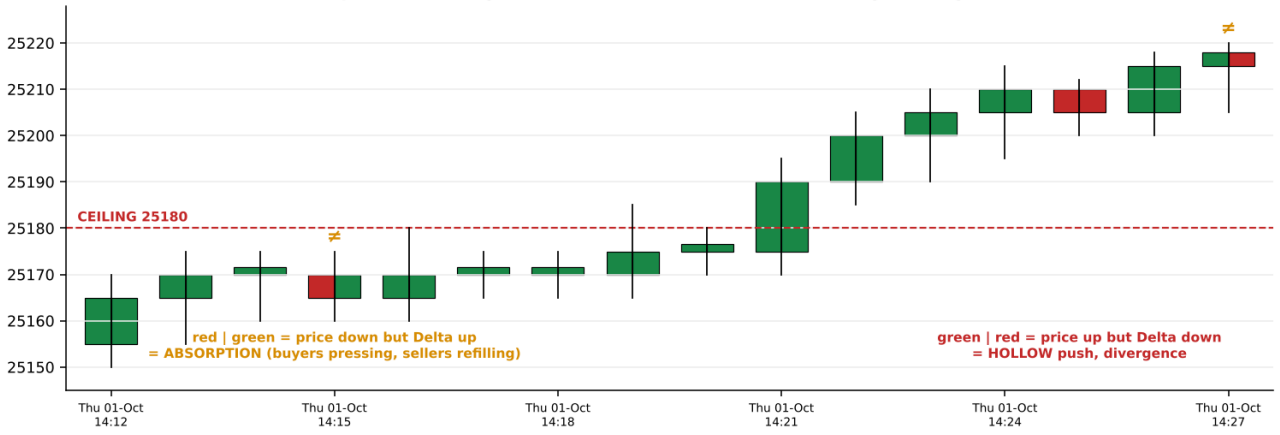
**CONCEPT · Order-book + order-flow read of a ceiling test (drawn paths, not real candles)**



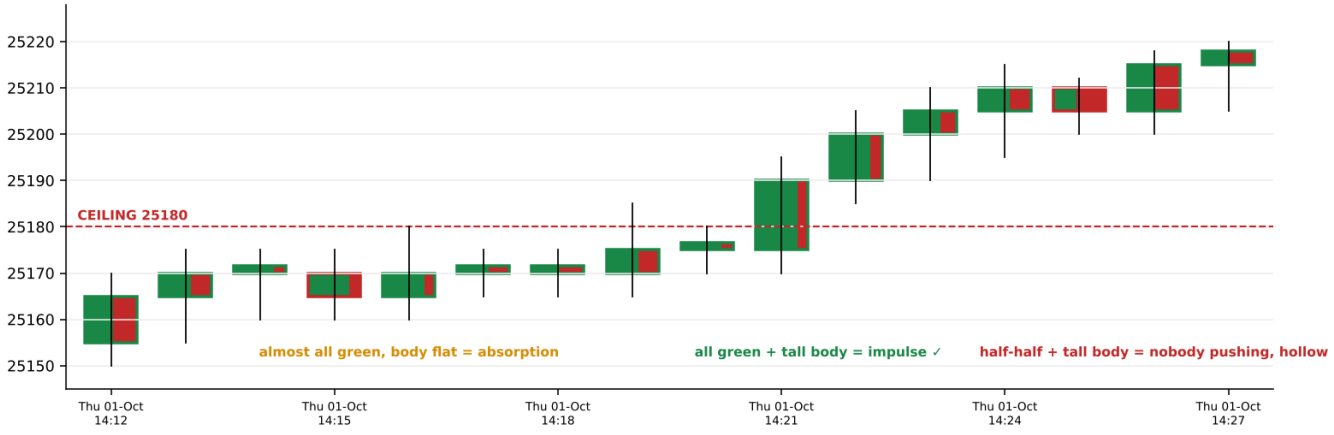

Panel 1 price vs CEILING/FLOOR · Panel 2 Delta bars (green = aggressive buying, red = aggressive selling) with CVD line · Panel 3 resting ASK qty vs BID qty, top-5 levels

**DELTA INSIDE THE CANDLE · NIFTY OCT FUT 1-min · Thu 01-Oct-2026 14:12–14:27 · SYNTHETIC**

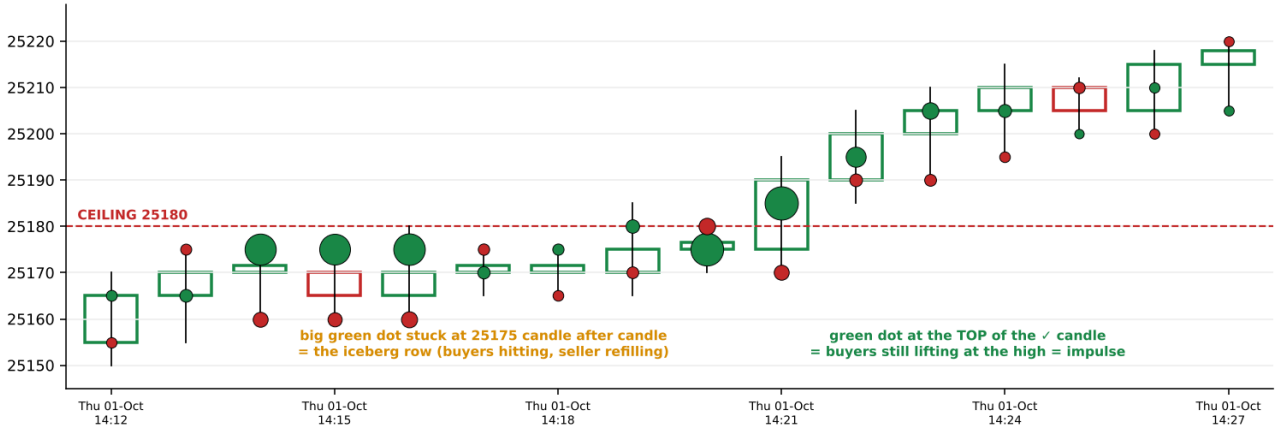
**A · SPLIT CANDLE  left half = price colour · right half = Delta colour  (two colours disagree = signal)**




**B · SHARE CANDLE  body split by buy-share (green) vs sell-share (red) · outline = price colour**

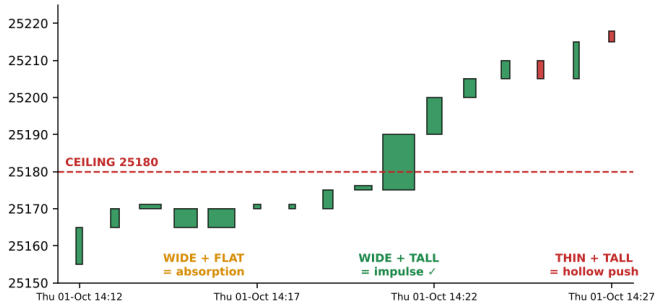



**C · DELTA POINTS  ● green = price row with highest Delta · ● red = row with lowest Delta  (where the fight was)**

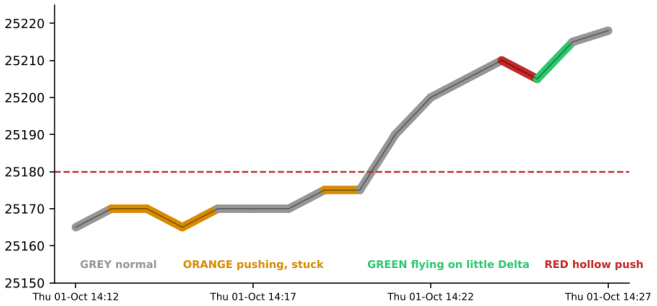


**INNOVATION MOCK-UPS · all derived from the same Fyers tick log · SYNTHETIC DATA**

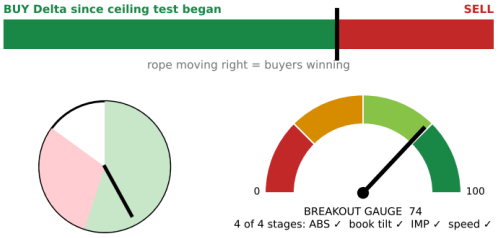
### 1 · PRESSURE GLYPH (width = push, height = progress)



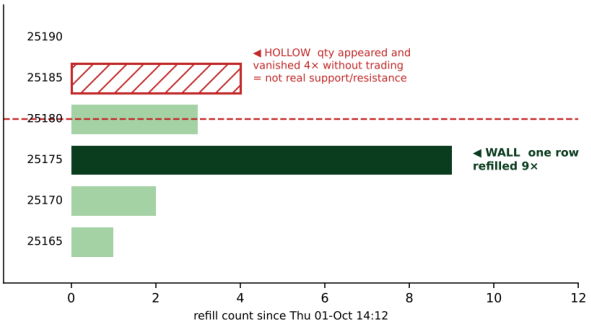

### 2 · EFFICIENCY HEAT LINE (points moved ÷ Delta)




### 3 · TUG-OF-WAR · ABSORPTION CLOCK · BREAKOUT GAUGE




### 4 · ICEBERG LADDER + SPOOF DETECTOR




### 5 · SIGNATURE SENTENCE + DNA STRIP (pattern = words, searchable)

ABS → ABS → EXH → START → ✓IMP

the whole ceiling test read as one sentence — memorise the sentence, not the numbers

`+L>H  +L>H  +S>H  +S=H  +L>L  +XL>L ✓`

each candle = 4 letters: Delta sign · Delta size (S/L/XL) · progress (>,=) · book tilt (H/L)

**search past logs for +S=H +L>L +XL>L ✓ → found 23 times, held 17 (74%)**

### 6 · PHASE-AWARE TAG · OPTIONS LAMP · TEACH MODE

ABS — in ACC parentage = fuel for RALLY UP

ABS — in DIST parentage = sellers defending

OPTIONS LAMP — ATM CE IV flat at ✓IMP → entry allowed

same signature, tag colour from PARENTAGE SCORE

**TEACH MODE Thu 01-Oct 14:17 future hidden**

What is this candle?

ABS EXH IMP DIV

# 16 · PYTHON TWIN — orderflow_engine.py WITH PLAIN-ENGLISH NOTES

## 16.1 · Block map

| # | Block | What it does in plain English | Spec section |
|---|---|---|---|
| 0 | CONFIG | One dictionary holding every number in §13 with its owner in a comment. Change a number here, nowhere else. | §13 |
| 1 | TickLogger | Writes every trade and every depth update to a daily JSONL file. Computes nothing. The comment under it shows how the Fyers callbacks are wired. | §3.6 |
| 2 | TickRule | For each trade decides who was aggressive by comparing the price to the best bid and ask. Remembers the last side for trades inside the spread. | §4.1 |
| 3 | Candle / Aggregator | Builds 1-minute candles from ticks. Each candle keeps buy volume, sell volume, trade count and the per-row buy/sell volumes. `delta`, `volume`, `tape_speed`, `row_delta()` read from them. `avg_abs_delta()` and `dot_ref()` are the two references. `dots()` returns the green and red dot rows and sizes. | §4.2–4.4 |
| 4 | Book | Keeps the latest depth and a 10-minute history of top-5 quantities. `tilt()`, `wall()`, `pulling()`, `stacking()` read the book; `update()` also counts refills (iceberg evidence) and vanishes (spoof evidence) per level, using how much actually traded at that price. | §4.5, §5.6–5.10 |
| 5 | Detectors | One function per signature: `det_ABS`, `det_EXH`, `det_IMP`, `det_STK`, `det_DIV`, `det_SPEED`. Each returns (fired, value). `warm()` blocks all of them for the first 5 candles. `BreakTracker` prints START and ✓ once each and re-arms on a close back inside. | §5, §2.5 |
| 6 | Witness | `witness()` gives one instrument's verdict UP / DOWN / QUIET; `two_witness()` applies the matrix. | §11 |
| 7 | AbsorptionClock | Starts on the first ABS, reports the zone, resets on ✓. | §4.7, §10.1 |
| 8 | running_state | Builds the five-line box. | §6.2 |
| 9 | tags_for_history / draw_main_chart | Re-evaluates every candle as it was seen live, then draws heat + hollow candles + dots + tags + the box, with day-date-time axis labels. | §6.1 |
| 10 | replay | Reads a logged file, runs everything in order, writes the chart. This is also the study harness. | §3.6, §14 |

## 16.2 · The source, verbatim

```
"""
PINAKA · ORDER FLOW ENGINE  (Python twin of Spec OF-1, draft 1, Sun 04-Oct-2026)

What this file does, in plain English:
  1. LOGGER      — records every trade tick and every depth update from Fyers to a daily file.
  2. CLASSIFIER  — tags each trade as buyer-aggressive or seller-aggressive (tick rule).
  3. AGGREGATOR  — rolls ticks into 1-minute candles with Delta, CVD, per-row Delta, tape speed.
  4. BOOK        — tracks the resting order book: top-N quantities, tilt, pulling/stacking,
                   refill counts (iceberg), hollow counts (spoof), the wall.
  5. DETECTORS   — the eight signatures (ABS, EXH, DIV, IMP, STK, PULL/STACK, ICE, SPEED)
                   plus the C/F BREAK START and ✓ states.
  6. WITNESS     — the two-witness rule for ATM CE and PE books.
  7. CHART       — draws the main chart (heat + dot candles + tags) and the running-state box.

Everything is driven by the CONFIG table below. Pine / Node.js are derived from THIS file.
"""

from __future__ import annotations
import json, math, time, datetime as dt
```

```
        from collections import deque, defaultdict
        from dataclasses import dataclass, field

        # ================================================================================
        # 0. CONFIGURATION  (mirrors Spec OF-1 §13 — every number has an owner)
        # ================================================================================
        CONFIG = {
            # --- data / feeds ---
            "SYMBOL_FUT":        "NSE:NIFTY26OCTFUT",   # Tushar  : current-month NIFTY futures
            "SYMBOLS_OPT":       [],                    # engine  : rolling ATM CE/PE, filled at run time
            "ATM_STEP":          50,                    # NSE     : NIFTY strike step
            "DEPTH_LEVELS_LOG":  50,                    # Tushar  : 50 = TBT feed, 5 = normal feed
            "DEPTH_LEVELS_HEAT": 10,                    # Claude  : rows drawn in the heat view (5 each side)
            # --- candle / rows ---
            "CANDLE_SEC":        60,                    # Tushar  : 1-minute candles
            "ROW_SIZE":          5.0,                   # Claude  : one price row = 5 NIFTY points
            # --- tick rule ---
            "MID_TRADE_RULE":    "last",                # Claude  : trade between bid/ask → same side as last
classified trade
            # --- dots ---
            "DOT_REF_WINDOW":    20,                    # Claude  : candles in the rolling reference
            "DOT_FLOOR":         0.3,                   # Claude  : below this × ref → no dot
            "DOT_MED":           1.0,                   # Claude  : ≥ this × ref → medium
            "DOT_LARGE":         2.0,                   # Claude  : ≥ this × ref → large
            # --- signatures ---
            "ABS_DELTA_X":       1.5,                   # Claude  : candle Delta ≥ 1.5 × avg |Delta| ...
            "ABS_PROGRESS_ROWS": 1,                     # Claude  : ... with |close-open| ≤ 1 row  = ABS
            "EXH_CANDLES":       2,                     # Claude  : Delta shrinking for 2 candles at same price
= EXH
            "EXH_SHRINK":        0.5,                   # Claude  : each smaller than the one before, last ≤ 50
% of first
            "IMP_DELTA_X":       3.0,                   # Claude  : candle Delta ≥ 3 × avg |Delta| AND progress
≥ 2 rows = IMP
            "IMP_PROGRESS_ROWS": 2,
            "STK_RATIO":         3.0,                   # Claude  : ask ≥ 3 × bid on a row = buy imbalance (or
reverse)
            "STK_ROWS":          3,                     # Claude  : ≥ 3 consecutive imbalanced rows = STK
            "DIV_LOOKBACK":      5,                     # Claude  : price higher high vs CVD lower high over 5
candles = DIV
            "ICE_REFILLS":       5,                     # Claude  : a row refilled ≥ 5 times while price stays =
ICE
            "ICE_REFILL_RATIO":  0.8,                   # Claude  : refill counts if qty returns to ≥ 80 % of
its earlier size
            "SPOOF_VANISH":      3,                     # Claude  : qty appears then vanishes untraded ≥ 3 times
= HOLLOW
            "SPOOF_MIN_QTY_X":   2.0,                   # Claude  : only levels ≥ 2 × median level qty count
            "PULL_DROP":         0.5,                   # Claude  : top-5 ask qty falls to ≤ 50 % of its 10-min
avg = PULL
            "STACK_RISE":        1.5,                   # Claude  : top-5 bid qty rises to ≥ 150 % of its 10-min
avg = STACK
            "BOOK_AVG_MIN":      10,                    # Claude  : minutes in the book-quantity average
            "WALL_X":            3.0,                   # Claude  : one level ≥ 3 × median of its side = WALL
            "SPEED_X":           2.0,                   # Claude  : trades/sec ≥ 2 × 10-min avg = SPEED
            "AVG_DELTA_WINDOW":  20,                    # Claude  : candles in avg |Delta|
            "WARMUP_CANDLES":    5,                     # Claude  : no signature prints before 5 completed
candles
            # --- time ---
            "ABS_CLOCK_GREEN_MIN": 3,                   # open    : ✓ window start (minutes under ceiling) — set
from study
            "ABS_CLOCK_RED_MIN":  12,                   # open    : failure zone start — set from study
            # --- break states (from Pinaka Spec) ---
            "BREAK_NEEDS_BODY_CLOSE": True,             # Tushar  : ✓ = full candle CLOSE beyond the band
        }

        # ================================================================================
        # 1. LOGGER  — writes raw ticks to disk. Nothing is computed here; computing comes later.
        # ================================================================================
        class TickLogger:
            """One line of JSON per message. File per day per symbol. Replayable."""
            def __init__(self, folder: str):
                self.folder = folder
                self.files = {}

            def _file(self, symbol: str):
                day = dt.date.today().strftime("%Y-%m-%d")
                key = f"{symbol}_{day}"
                if key not in self.files:
                    safe = symbol.replace(":", "_")
                    self.files[key] = open(f"{self.folder}/{safe}_{day}.jsonl", "a")
                return self.files[key]

            def on_trade(self, symbol: str, ts: float, ltp: float, ltq: int, bid: float, ask: float):
```

```
                # ts = epoch seconds from the feed; bid/ask = best quotes at that instant
                self._file(symbol).write(json.dumps({"t": "T", "ts": ts, "p": ltp, "q": ltq, "b": bid, "a":
ask}) + "\n")

            def on_depth(self, symbol: str, ts: float, bids: list, asks: list):
                # bids/asks = list of [price, qty] from best outwards, up to DEPTH_LEVELS_LOG levels
                self._file(symbol).write(json.dumps({"t": "D", "ts": ts, "bids": bids, "asks": asks}) + "\n")

            def flush(self):
                for f in self.files.values():
                    f.flush()

        # How the logger is wired to Fyers (see Spec §3 for the setup steps):
        #   from fyers_apiv3.FyersWebsocket import data_ws        # 5-level feed: SymbolUpdate + DepthUpdate
        #   from fyers_apiv3.FyersWebsocket import tbt_ws         # 50-level feed (protobuf): depth only
        #   on every SymbolUpdate  -> logger.on_trade(symbol, ts, ltp, last_traded_qty, bid_price1, ask_price1)
        #   on every DepthUpdate   -> logger.on_depth(symbol, ts, bids_5, asks_5)
        #   on every TBT depth msg -> logger.on_depth(symbol, ts, bids_50, asks_50)

        # ============================================================================
        # 2. CLASSIFIER — tick rule. Who was aggressive in this trade?
        # ============================================================================
        class TickRule:
            def __init__(self):
                self.last_side = +1   # +1 = buyer aggressive, -1 = seller aggressive

            def side(self, price: float, bid: float, ask: float) -> int:
                if price >= ask:      # paid the offer → buyer was aggressive
                    self.last_side = +1
                elif price <= bid:    # hit the bid → seller was aggressive
                    self.last_side = -1
                else:                 # traded inside the spread → keep the previous side (CONFIG
MID_TRADE_RULE)
                    pass
                return self.last_side

        # ============================================================================
        # 3. AGGREGATOR — ticks → 1-minute candles carrying order-flow fields
        # ============================================================================
        @dataclass
        class Candle:
            start: int                        # epoch seconds of candle open
            open: float; high: float; low: float; close: float
            buy_vol: int = 0; sell_vol: int = 0
            trades: int = 0
            row_buy: dict = field(default_factory=lambda: defaultdict(int))   # price-row → buy volume
            row_sell: dict = field(default_factory=lambda: defaultdict(int))  # price-row → sell volume

            @property
            def delta(self): return self.buy_vol - self.sell_vol
            @property
            def volume(self): return self.buy_vol + self.sell_vol
            @property
            def tape_speed(self): return self.trades / CONFIG["CANDLE_SEC"]      # trades per second
            def row_delta(self): return {r: self.row_buy[r] - self.row_sell[r] for r in set(self.row_buy) |
set(self.row_sell)}

        def row_of(price: float) -> float:
            """Which 5-point row does this price belong to?  25177 → 25175."""
            s = CONFIG["ROW_SIZE"]
            return math.floor(price / s) * s

        class Aggregator:
            def __init__(self):
                self.rule = TickRule()
                self.candles: list[Candle] = []
                self.cvd = 0

            def on_trade(self, ts: float, price: float, qty: int, bid: float, ask: float):
                start = int(ts // CONFIG["CANDLE_SEC"]) * CONFIG["CANDLE_SEC"]
                if not self.candles or self.candles[-1].start != start:
                    self.candles.append(Candle(start, price, price, price, price))
                c = self.candles[-1]
                c.high = max(c.high, price); c.low = min(c.low, price); c.close = price; c.trades += 1
                side = self.rule.side(price, bid, ask)
                r = row_of(price)
                if side > 0:
                    c.buy_vol += qty; c.row_buy[r] += qty; self.cvd += qty
                else:
                    c.sell_vol += qty; c.row_sell[r] += qty; self.cvd -= qty

            def avg_abs_delta(self, n=None):
```

```
                n = n or CONFIG["AVG_DELTA_WINDOW"]
                xs = [abs(c.delta) for c in self.candles[-n-1:-1]] or [1]
                return max(sum(xs) / len(xs), 1)

            def dot_ref(self):
                """Reference for dot size = average of each candle's biggest |row-Delta| over the window."""
                n = CONFIG["DOT_REF_WINDOW"]
                xs = []
                for c in self.candles[-n-1:-1]:
                    rd = c.row_delta()
                    if rd: xs.append(max(abs(v) for v in rd.values()))
                return max(sum(xs) / len(xs), 1) if xs else 1

            def dots(self, c: Candle):
                """Returns (green_row, green_size, red_row, red_size); size in {None,'S','M','L'}."""
                rd = c.row_delta()
                if not rd: return None, None, None, None
                ref = self.dot_ref()
                def size(v):
                    r = abs(v) / ref
                    if r < CONFIG["DOT_FLOOR"]: return None
                    if r < CONFIG["DOT_MED"]:   return "S"
                    if r < CONFIG["DOT_LARGE"]: return "M"
                    return "L"
                gmax = max(rd, key=rd.get); rmin = min(rd, key=rd.get)
                g = (gmax, size(rd[gmax])) if rd[gmax] > 0 else (None, None)
                r = (rmin, size(rd[rmin])) if rd[rmin] < 0 else (None, None)
                return g[0], g[1], r[0], r[1]

        # ================================================================================
        # 4. BOOK — the resting order book over time
        # ================================================================================
        class Book:
            def __init__(self):
                self.ts = 0
                self.bids: list = []      # [[price, qty], ...] best first
                self.asks: list = []
                self.hist_bid5 = deque(maxlen=CONFIG["BOOK_AVG_MIN"] * 60)   # one sample per second
                self.hist_ask5 = deque(maxlen=CONFIG["BOOK_AVG_MIN"] * 60)
                self.refill = defaultdict(int)       # price → times it refilled after being eaten (iceberg
evidence)
                self.vanish = defaultdict(int)       # price → times a big qty appeared and vanished untraded
(spoof evidence)
                self._seen_big = {}                  # price → qty when it was last big
                self._traded_at = defaultdict(int)   # price → volume traded there since it was big

            def top_qty(self, side: str, n=5):
                lv = self.bids if side == "bid" else self.asks
                return sum(q for _, q in lv[:n])

            def tilt(self):
                a = self.top_qty("ask") or 1
                return self.top_qty("bid") / a            # > 1 = book tilted to buyers

            def wall(self, side: str):
                """Biggest level on one side if it is ≥ WALL_X × that side's median. Returns (price, qty) or
None."""
                lv = self.bids if side == "bid" else self.asks
                if len(lv) < 3: return None
                qs = sorted(q for _, q in lv); med = qs[len(qs) // 2] or 1
                p, q = max(lv, key=lambda x: x[1])
                return (p, q) if q >= CONFIG["WALL_X"] * med else None

            def update(self, ts: float, bids: list, asks: list, traded_rows: dict | None = None):
                """traded_rows = {price: qty traded since last update} — needed to tell eaten from pulled."""
                traded_rows = traded_rows or {}
                prev = {p: q for p, q in self.bids + self.asks}
                allq = sorted(q for _, q in bids + asks) or [1]
                med = allq[len(allq) // 2] or 1
                for p, q in bids + asks:
                    # (a) ICEBERG: level was eaten (traded) and the qty came back to ≥ 80 % of its old size
                    if p in self._seen_big and self._traded_at[p] >= CONFIG["ICE_REFILL_RATIO"] *
self._seen_big[p] \
                            and q >= CONFIG["ICE_REFILL_RATIO"] * self._seen_big[p]:
                        self.refill[p] += 1; self._traded_at[p] = 0
                    # (b) remember big levels
                    if q >= CONFIG["SPOOF_MIN_QTY_X"] * med:
                        self._seen_big[p] = q
                for p, q in prev.items():
                    # (c) SPOOF: a big level disappeared although (almost) nothing traded there
                    if p in self._seen_big and all(p != x[0] for x in bids + asks) \
                            and self._traded_at[p] < 0.2 * self._seen_big[p]:
```

```
                        self.vanish[p] += 1; del self._seen_big[p]
                for p, q in traded_rows.items():
                    self._traded_at[p] += q
                self.bids, self.asks, self.ts = bids, asks, ts
                self.hist_bid5.append(self.top_qty("bid")); self.hist_ask5.append(self.top_qty_ex_wall("ask"))

            def top_qty_ex_wall(self, side: str, n=5):
                """Top-n qty with the wall level (if any) left out — pulling is about the levels ABOVE the
wall."""
                lv = self.bids if side == "bid" else self.asks
                w = self.wall(side)
                lv = [x for x in lv if not (w and x[0] == w[0])]
                return sum(q for _, q in lv[:n])

            def pulling(self):
                """Top-5 ASK qty (wall excluded) ≤ PULL_DROP × its 10-min average → asks being pulled."""
                if len(self.hist_ask5) < 60: return False
                avg = sum(self.hist_ask5) / len(self.hist_ask5) or 1
                return self.top_qty_ex_wall("ask") <= CONFIG["PULL_DROP"] * avg

            def stacking(self):
                """Top-5 BID qty ≥ STACK_RISE × its 10-min average → bids being stacked."""
                if len(self.hist_bid5) < 60: return False
                avg = sum(self.hist_bid5) / len(self.hist_bid5) or 1
                return self.top_qty("bid") >= CONFIG["STACK_RISE"] * avg

            def iceberg_rows(self):
                return [p for p, n in self.refill.items() if n >= CONFIG["ICE_REFILLS"]]

            def hollow_rows(self):
                return [p for p, n in self.vanish.items() if n >= CONFIG["SPOOF_VANISH"]]

        # ================================================================================
        # 5. DETECTORS — one function per signature. Each returns True/False and the number that fired it.
        # ================================================================================
        def rows_moved(c: Candle):
            return abs(c.close - c.open) / CONFIG["ROW_SIZE"]

        def warm(agg: Aggregator): return len(agg.candles) > CONFIG["WARMUP_CANDLES"]

        def det_ABS(agg: Aggregator):
            """ABSORPTION: big Delta, no progress."""
            if not warm(agg): return False, 0
            c = agg.candles[-1]; ref = agg.avg_abs_delta()
            fired = abs(c.delta) >= CONFIG["ABS_DELTA_X"] * ref and rows_moved(c) <= CONFIG["ABS_PROGRESS_ROWS"]
            return fired, c.delta / ref

        def det_EXH(agg: Aggregator):
            """EXHAUSTION: Delta shrinking candle after candle at the same price."""
            n = CONFIG["EXH_CANDLES"] + 1
            if len(agg.candles) < n: return False, 0
            cs = agg.candles[-n:]
            same_price = all(abs(x.close - cs[0].close) <= CONFIG["ROW_SIZE"] for x in cs)
            shrinking = all(abs(cs[i].delta) < abs(cs[i-1].delta) for i in range(1, n)) \
                        and abs(cs[-1].delta) <= CONFIG["EXH_SHRINK"] * abs(cs[0].delta)
            same_sign = all((x.delta > 0) == (cs[0].delta > 0) for x in cs)
            return same_price and shrinking and same_sign, abs(cs[-1].delta) / max(abs(cs[0].delta), 1)

        def det_IMP(agg: Aggregator):
            """IMPULSE: Delta ≥ 3 × average AND price actually moved."""
            if not warm(agg): return False, 0
            c = agg.candles[-1]; ref = agg.avg_abs_delta()
            fired = abs(c.delta) >= CONFIG["IMP_DELTA_X"] * ref and rows_moved(c) >= CONFIG["IMP_PROGRESS_ROWS"]
            return fired, c.delta / ref

        def det_STK(c: Candle):
            """STACKED IMBALANCE: ≥ 3 consecutive rows where one side is ≥ 3 × the other."""
            rows = sorted(set(c.row_buy) | set(c.row_sell))
            best_run, run, direction = 0, 0, 0
            for r in rows:
                b, s = c.row_buy[r], c.row_sell[r]
                d = +1 if b >= CONFIG["STK_RATIO"] * max(s, 1) else (-1 if s >= CONFIG["STK_RATIO"] * max(b, 1)
else 0)
                run = run + 1 if (d != 0 and d == direction) else (1 if d != 0 else 0)
                direction = d if d != 0 else direction
                best_run = max(best_run, run)
            return best_run >= CONFIG["STK_ROWS"], best_run

        def det_DIV(agg: Aggregator, cvd_series: list):
            """DIVERGENCE: price makes a higher high while CVD makes a lower high (or the mirror)."""
            n = CONFIG["DIV_LOOKBACK"]
            if len(agg.candles) < 2 * n: return False, 0
```

```
            now, before = agg.candles[-n:], agg.candles[-2*n:-n]
            cvd_now, cvd_before = cvd_series[-n:], cvd_series[-2*n:-n]
            R = CONFIG["ROW_SIZE"]
            bear = max(c.high for c in now) >= max(c.high for c in before) + R and max(cvd_now) <
max(cvd_before)
            bull = min(c.low for c in now)  <= min(c.low for c in before)  - R and min(cvd_now) >
min(cvd_before)
            return bear or bull, (-1 if bear else (+1 if bull else 0))

        def det_SPEED(agg: Aggregator):
            """TAPE SPEED: trades/sec ≥ 2 × the 10-minute average."""
            n = CONFIG["BOOK_AVG_MIN"]
            if len(agg.candles) < n + 1: return False, 0
            avg = sum(c.tape_speed for c in agg.candles[-n-1:-1]) / n or 1e-9
            return agg.candles[-1].tape_speed >= CONFIG["SPEED_X"] * avg, agg.candles[-1].tape_speed / avg

        class BreakTracker:
            """C BREAK START / C BREAK ✓ / F BREAK START / F BREAK ✓ per Pinaka Spec.
               START = price gets beyond the band at all (a wick poke counts) — printed ONCE.
               ✓     = a full candle CLOSE beyond it (green above ceiling, red below floor) — printed ONCE.
               After ✓ the state is ABOVE / BELOW until price closes back inside, which re-arms the tracker."""
            def __init__(self): self.c_start = self.c_ok = self.f_start = self.f_ok = False
            def update(self, c: Candle, ceiling, floor):
                out = []
                if ceiling is not None:
                    if c.close <= ceiling and self.c_ok: self.c_start = self.c_ok = False      # closed back
inside → re-arm
                    if c.high > ceiling and not self.c_start: self.c_start = True; out.append("C BREAK START")
                    if c.close > ceiling and c.close > c.open and not self.c_ok: self.c_ok = True; out.append("C
BREAK ✓")
                if floor is not None:
                    if c.close >= floor and self.f_ok: self.f_start = self.f_ok = False
                    if c.low < floor and not self.f_start: self.f_start = True; out.append("F BREAK START")
                    if c.close < floor and c.close < c.open and not self.f_ok: self.f_ok = True; out.append("F
BREAK ✓")
                return out
            def state(self, c: Candle, ceiling, floor):
                if ceiling is not None and self.c_ok: return "ABOVE CEILING"
                if floor is not None and self.f_ok: return "BELOW FLOOR"
                if self.c_start: return "C BREAK START seen, no ✓"
                if self.f_start: return "F BREAK START seen, no ✓"
                return "inside band"

        # ================================================================================
        # 6. TWO-WITNESS RULE — ATM CE book and ATM PE book read separately, then compared
        # ================================================================================
        def witness(agg: Aggregator, book: Book) -> str:
            """One instrument's verdict: 'UP', 'DOWN' or 'QUIET'."""
            c = agg.candles[-1]; ref = agg.avg_abs_delta()
            if c.delta >= 1.0 * ref and (book.pulling() or rows_moved(c) >= 1): return "UP"      # aggressive
buying
            if c.delta <= -1.0 * ref and (book.stacking() is False or rows_moved(c) >= 1): return "DOWN"  #
aggressive selling
            return "QUIET"

        def two_witness(ce: str, pe: str) -> str:
            """CE buyers aggressive + PE being dumped = UP. Mirror = DOWN. Both bought = hedged. Else none."""
            if ce == "UP"   and pe == "DOWN": return "UP CONFIRMED"
            if ce == "DOWN" and pe == "UP":   return "DOWN CONFIRMED"
            if ce == "UP"   and pe == "UP":   return "HEDGED FLOW — NO TRADE"
            if ce == "DOWN" and pe == "DOWN": return "UNWIND — NO TRADE"
            return "NO WITNESS"

        # ================================================================================
        # 7. ABSORPTION CLOCK — minutes price has sat under the ceiling since the first ABS tag
        # ================================================================================
        class AbsorptionClock:
            def __init__(self): self.start = None
            def update(self, abs_fired: bool, ts: int):
                if abs_fired and self.start is None: self.start = ts
                if not abs_fired and self.start is not None and False: pass   # the clock resets only on a break
(see reset)
                return self.minutes(ts)
            def minutes(self, ts: int): return 0 if self.start is None else (ts - self.start) / 60
            def zone(self, ts: int):
                m = self.minutes(ts)
                if self.start is None: return "OFF"
                if m < CONFIG["ABS_CLOCK_GREEN_MIN"]: return "EARLY"
                if m < CONFIG["ABS_CLOCK_RED_MIN"]:   return "GREEN"
                return "RED"
            def reset(self): self.start = None
```

```
        # ================================================================================
        # 8. RUNNING STATE — the only text the trader reads live (legibility first)
        # ================================================================================
        def running_state(agg: Aggregator, book: Book, clock: AbsorptionClock, brk: "BreakTracker", ceiling,
floor, ce_w="QUIET", pe_w="QUIET"):
            c = agg.candles[-1]; ts = c.start
            stamp = dt.datetime.fromtimestamp(ts).strftime("%a %d-%b %H:%M")
            tags = []
            if det_ABS(agg)[0]: tags.append("ABS")
            if det_EXH(agg)[0]: tags.append("EXH")
            if det_IMP(agg)[0]: tags.append("IMP")
            if det_STK(c)[0]:   tags.append("STK")
            lines = [f"RUNNING STATE  {stamp}"]
            if ceiling: lines.append(f"1. Under CEILING {ceiling:.0f}: " + (" → ".join(tags) if tags else
"nothing yet"))
            bk = ("ASK thin" if book.pulling() else "ASK firm") + " / " + ("BID stacked" if book.stacking() else
"BID normal") + f", tilt {book.tilt():.1f}"
            lines.append(f"2. Book: {bk}")
            lines.append("3. " + brk.state(c, ceiling, floor))
            lines.append(f"4. Witness: CE {ce_w} / PE {pe_w} → {two_witness(ce_w, pe_w)}")
            lines.append(f"5. Clock: {clock.zone(ts)} ({clock.minutes(ts):.0f} min)")
            return "\n".join(lines)

        # ================================================================================
        # 9. CHART — main chart = heat (resting book) + dot candles + tags.  Debug board separate.
        # ================================================================================
        def tags_for_history(agg: Aggregator, cvd_series: list):
            """Re-evaluate every completed candle as it would have been seen live. Returns {index: [tags]}."""
            out, brk = {}, BreakTracker()
            for i in range(1, len(agg.candles) + 1):
                tmp = Aggregator(); tmp.candles = agg.candles[:i]; c = tmp.candles[-1]; t = []
                if det_ABS(tmp)[0]: t.append("ABS")
                if det_EXH(tmp)[0]: t.append("EXH")
                if det_IMP(tmp)[0]: t.append("IMP")
                if det_STK(c)[0]:   t.append("STK")
                if det_DIV(tmp, cvd_series[:i])[0]: t.append("DIV")
                if det_SPEED(tmp)[0]: t.append("SPEED")
                out[i - 1] = t
            return out

        TAG_COLOUR = {"ABS": "#d98e04", "EXH": "#d98e04", "IMP": "#1b8a4a", "STK": "#1b8a4a", "DIV": "#c62828",
"SPEED": "#555",
                      "C BREAK START": "black", "C BREAK ✓": "#1b8a4a", "F BREAK START": "black", "F BREAK ✓":
"#c62828"}

        def draw_main_chart(agg: Aggregator, book_snapshots: list, ceiling, floor, out_png: str, state_text:
str,
                            tags: dict | None = None, break_events: dict | None = None):
            """book_snapshots: list of (ts, bids, asks) at ~10-second spacing, used for the heat background.
               tags: {candle index: [signature tags]}; break_events: {candle index: [START/✓ events]}."""
            import numpy as np, matplotlib; matplotlib.use("Agg")
            import matplotlib.pyplot as plt
            from matplotlib.patches import Rectangle
            from matplotlib.colors import LinearSegmentedColormap
            GREEN, RED, AMB = "#1b8a4a", "#c62828", "#d98e04"
            cs = agg.candles[-60:]
            if not cs: return
            lo = min(c.low for c in cs) - 4 * CONFIG["ROW_SIZE"]; hi = max(c.high for c in cs) + 4 *
CONFIG["ROW_SIZE"]
            rows = np.arange(row_of(lo), row_of(hi) + CONFIG["ROW_SIZE"], CONFIG["ROW_SIZE"])
            t0 = cs[0].start; t1 = cs[-1].start + CONFIG["CANDLE_SEC"]
            snaps = [s for s in book_snapshots if t0 <= s[0] <= t1] or [(t0, [], [])]
            T = max(len(snaps), 1)
            heat = np.zeros((len(rows), T)); side = np.zeros((len(rows), T))
            for j, (ts, bids, asks) in enumerate(snaps):
                allq = [q for _, q in bids[:CONFIG["DEPTH_LEVELS_HEAT"]] + asks[:CONFIG["DEPTH_LEVELS_HEAT"]]]
                mx = max(allq) if allq else 1
                for p, q in bids[:CONFIG["DEPTH_LEVELS_HEAT"]]:
                    k = int((row_of(p) - rows[0]) / CONFIG["ROW_SIZE"])
                    if 0 <= k < len(rows): heat[k, j] = max(heat[k, j], q / mx); side[k, j] = +1
                for p, q in asks[:CONFIG["DEPTH_LEVELS_HEAT"]]:
                    k = int((row_of(p) - rows[0]) / CONFIG["ROW_SIZE"])
                    if 0 <= k < len(rows): heat[k, j] = max(heat[k, j], q / mx); side[k, j] = -1
            cm_b = LinearSegmentedColormap.from_list("b", ["white", "#a5d6a7", "#0b3d1f"])
            cm_a = LinearSegmentedColormap.from_list("a", ["white", "#ef9a9a", "#7f0000"])
            img = np.ones((len(rows), T, 4))
            for k in range(len(rows)):
                for j in range(T):
                    img[k, j] = (cm_b if side[k, j] > 0 else cm_a)(heat[k, j] * 0.75)   # 75 % strength so dots
stay visible
            fig, ax = plt.subplots(figsize=(15, 8))
            ax.imshow(img, aspect="auto", origin="lower", extent=(-0.5, len(cs) - 0.5, rows[0] - 2.5, rows[-1] +
```

```
2.5), interpolation="nearest")
            for i, c in enumerate(cs):
                pc = GREEN if c.close >= c.open else RED
                ax.plot([i, i], [c.low, c.high], color="black", lw=1)
                ax.add_patch(Rectangle((i - 0.3, min(c.open, c.close)), 0.6, max(abs(c.close - c.open), 1.5),
fc="white", ec=pc, lw=2))
                gr, gs, rr, rs = agg.dots(c)
                ms = {"S": 7, "M": 12, "L": 18}
                if gs: ax.plot(i, gr + 2.5, "o", color=GREEN, ms=ms[gs], mec="black", mew=0.6, zorder=5)
                if rs: ax.plot(i, rr + 2.5, "o", color=RED, ms=ms[rs], mec="black", mew=0.6, zorder=5)
                k0 = len(agg.candles) - len(cs) + i          # index of this candle in the full list
                labels = (break_events or {}).get(k0, []) + (tags or {}).get(k0, [])   # break events first
                for n_, lab in enumerate(labels[:3]):        # legibility: at most 3 labels per candle
                    ax.text(i, c.high + 3 + n_ * 7, lab, ha="center", va="bottom", fontsize=7, weight="bold",
color="white",
                            bbox=dict(boxstyle="round,pad=0.2", fc=TAG_COLOUR.get(lab, "#555"), ec="none"))
            if ceiling: ax.axhline(ceiling, color=RED, ls="--", lw=1.2)
            if floor:   ax.axhline(floor, color="#1e4fa3", ls="--", lw=1.2)
            ax.text(0.012, 0.97, state_text, transform=ax.transAxes, va="top", fontsize=8.5, family="monospace",
                    bbox=dict(boxstyle="round", fc="white", ec="#999"))
            ticks = list(range(0, len(cs), max(1, len(cs) // 6)))
            ax.set_xticks(ticks); ax.set_xticklabels([dt.datetime.fromtimestamp(cs[i].start).strftime("%a %d-%b
%H:%M") for i in ticks], fontsize=8)
            ax.set_title(f"{CONFIG['SYMBOL_FUT']} · 1-min · heat = resting book · dots = where Delta peaked",
loc="left", weight="bold")
            fig.savefig(out_png, dpi=140, bbox_inches="tight"); plt.close(fig)

        # ================================================================================
        # 10. REPLAY — run the whole engine over a logged day (this is also the study harness)
        # ================================================================================
        def replay(jsonl_path: str, ceiling=None, floor=None, out_png=None):
            agg, book, clock, brk = Aggregator(), Book(), AbsorptionClock(), BreakTracker()
            cvd_series, snaps, last_snap, traded, done, events = [], [], 0, defaultdict(int), 0, {}
            with open(jsonl_path) as f:
                for line in f:
                    m = json.loads(line)
                    if m["t"] == "T":
                        agg.on_trade(m["ts"], m["p"], m["q"], m["b"], m["a"]); traded[m["p"]] += m["q"]
                        if len(cvd_series) < len(agg.candles): cvd_series.append(agg.cvd)
                        else: cvd_series[-1] = agg.cvd
                    else:
                        book.update(m["ts"], m["bids"], m["asks"], traded); traded.clear()
                        if m["ts"] - last_snap >= 10: snaps.append((m["ts"], m["bids"], m["asks"])); last_snap =
m["ts"]
                        if len(agg.candles) > done + 1:                       # a candle just completed → evaluate
it once
                            done = len(agg.candles) - 1
                            cc = agg.candles[done - 1]
                            ev = brk.update(cc, ceiling, floor)
                            if ev: events[done - 1] = ev
                            clock.update(det_ABS(agg)[0], cc.start)
                            if brk.c_ok or brk.f_ok: clock.reset()
            state = running_state(agg, book, clock, brk, ceiling, floor) if agg.candles else "no data"
            if out_png and agg.candles:
                draw_main_chart(agg, snaps, ceiling, floor, out_png, state, tags_for_history(agg, cvd_series),
events)
            return agg, book, clock, state

        if __name__ == "__main__":
            import sys
            agg, book, clock, state = replay(sys.argv[1], ceiling=float(sys.argv[2]) if len(sys.argv) > 2 else
None,
                                             out_png=sys.argv[3] if len(sys.argv) > 3 else None)
            print(state)
```

## 16.3 · How to run it

1. Install once: `pip install fyers-apiv3 matplotlib numpy`.
2. Log a day: wire the three callbacks shown under block 1 into the Fyers sockets; keep the process running 09:15–15:30.
3. Replay and draw: `python orderflow_engine.py NSE_NIFTY26OCTFUT_2026-10-01.jsonl 25180 chart.png` — the second argument is the ceiling from the Pinaka engine; the chart and the running-state text are produced.
4. Study: loop step 3 over every file; collect the per-candle rows (§14 point 2).

---

## 17 · OPEN POINTS

1. ABS_CLOCK_GREEN_MIN and ABS_CLOCK_RED_MIN — deferred to the study (§14 point 7). Starting values 3 and 12 are placeholders and are marked OPEN in §13.
2. Fyers field names on the TBT protobuf — to be confirmed against the SDK at connection time (§3.1). No effect on any rule.
3. The Fig 5 candidate layers (pressure glyph, efficiency heat line, gauge, DNA strip, options lamp, teach mode) — deferred until the main chart has run 20 sessions; none is required by any rule in this spec.
4. No other open points. Every rule in §4–§11 is closed with a value and an owner in §13.

---

# 18 · CHANGE REGISTER

| # | Draft | Date | Change |
|---|---|---|---|
| 1 | 1 | Sun 04-Oct-2026 | First issue. Captures the research of Sun 04-Oct-2026 15:26–21:50: order-book nomenclature; Fyers 5-level and 50-level TBT setup; tick rule, Delta, CVD, row-Delta, three-size dots; eight signatures plus WALL and HOLLOW; main chart = heat + dot candles + tags (Fig 8), running-state box, live ladder, debug board; heat cases H1–H8, dot cases D1–D8, combined sequences C1–C3; time strips; two-witness rule on ATM CE/PE; label list; configuration table; 20-session study protocol; Python twin `orderflow_engine.py` tested on a synthetic logged day (Fig 9). |